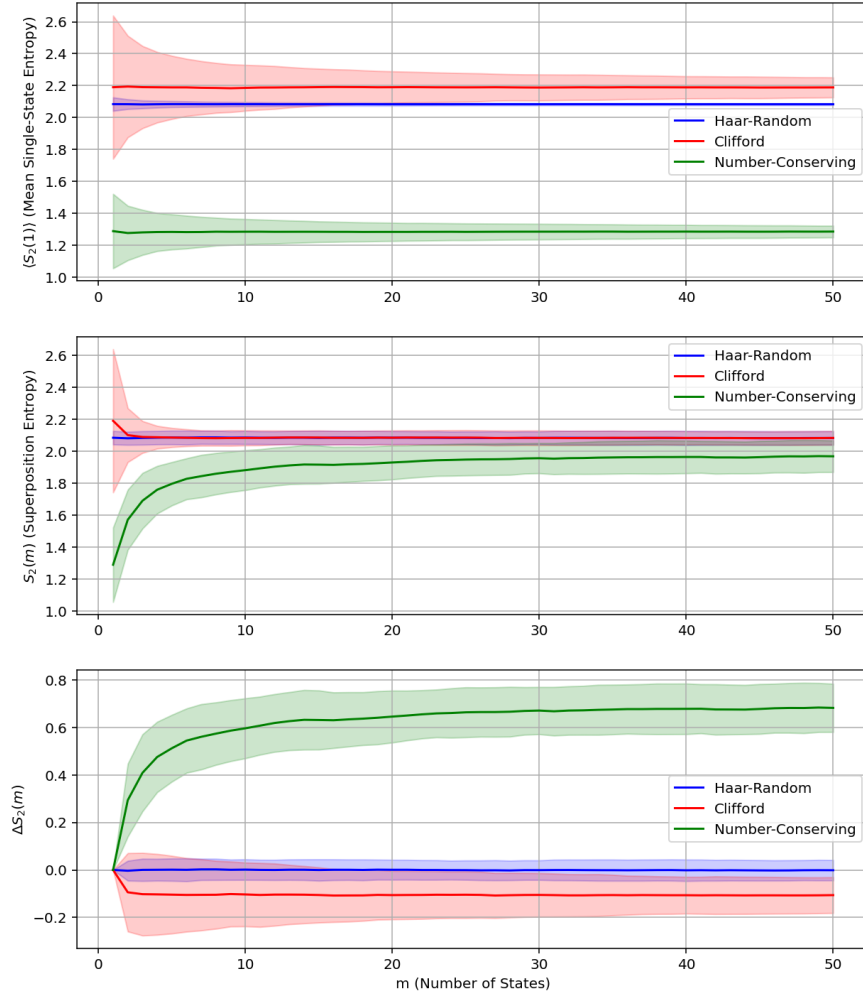


Entanglement in Superpositions of Quantum Many-Body States

Ben Lehany-Fee

Department of Physics, National University of Ireland Maynooth, Ireland.



Abstract

This project investigates how entanglement behaves for superpositions of Haar-random states, Clifford stabilizer states and number-conserving states. We compare the von Neumann and Rényi-2 entropies via Page curve analysis, and study how the Rényi-2 entropy changes as we superpose states. Our results show that Haar-random and Clifford stabilizer states show near-maximal entanglement, with superpositions of Haar-random states showing little change, and superposition of Clifford stabilizer states approaching the Haar-random entanglement entropy. Number-conserving states show sub-maximal entanglement, and their entropy increases on average when superposed.

1 Introduction:

In 1935, Einstein, Podolsky, and Rosen argued that quantum mechanics was incomplete. They found that for particles which had interacted and were now spatially separated, measuring the position or momentum of one particle allowed you to find the corresponding quantity of the other with certainty [1]. These correlations broke local realism and had no classical equivalent. However, rather than viewing these as a flaw in the quantum-mechanical model, Schrödinger instead recognised the importance of these correlations outright, coining the term *entanglement* to describe them in his 1935 paper:

”I would not call this one but rather the characteristic trait of quantum mechanics, the one that enforces its entire departure from classical lines of thought [2].”

This seeming conflict remained unsolved for decades. In 1952, Bohm proposed a non-local hidden-variable theory to try to address this paradox [3]. However, it was John Bell who would provide the missing piece of this puzzle. In 1964, he derived the Bell inequalities, a set of inequalities which he proved were violated by any local hidden-variable theory, yet were consistent with Bohm’s non-local theory [4]. These violations were later verified experimentally by Aspect in 1981 [5], and definitively proved the existence of quantum entanglement as a physical concept. This would lead to Aspect, Clauser and Zeilinger winning the 2022 Nobel Prize in Physics [6].

Today, quantum entanglement is a cornerstone of both many-body physics and quantum information science. It explains the interactions of particles across large distances, and allows us to study the behaviour of particles when affected by their surrounding environment. It also enables revolutionary technologies such as quantum computers, quantum encryption and quantum communication, which possess the potential to revolutionize certain aspects of modern-day technology.

While the entanglement of individual quantum states is well-documented for many classes of states, the entanglement of superpositions of quantum states is often more subtle. For instance, the Bell states have maximum entanglement, yet are superpositions of unentangled product states. This project investigates the Rényi-2 and von Neumann entropies of equal-weight superpositions for three key classes of states; Haar-random states, Clifford stabilizer states and number-conserving states. In Section 2, we discuss the preliminary theory of this project. In Section 3, we explain the computational implementations needed for this project, and we will present and analyse our results in Section 4.

2 Theory:

2.1 The Partial Trace & Entanglement Entropy:

This section introduces the key concepts required to quantify entanglement in quantum systems. We begin by explaining the concept of entanglement itself.

Consider a closed quantum system of N qubits. We can split the system into two disjoint subsystems, namely subsystem A with N_A qubits and Hilbert space $\mathcal{H}_A$, and subsystem B with $N_B = N - N_A$ qubits and Hilbert space $\mathcal{H}_B$. Note that our chosen qubits are not necessarily neighbours, but instead can be any arbitrary bipartition of the overall system. The Hilbert space of the overall system will hence be

$$\mathcal{H} = \mathcal{H}_A \otimes \mathcal{H}_B \quad (1)$$

A given state $|\psi\rangle \in \mathcal{H}$ is said to be a *product state* if the overall state can be written as a tensor product of states from the Hilbert spaces of the subsystems, i.e. $\exists |\psi_A\rangle \in \mathcal{H}_A, |\psi_B\rangle \in \mathcal{H}_B$ such that $|\psi\rangle = |\psi_A\rangle \otimes |\psi_B\rangle$. If this is not possible, it is instead an *entangled state* [7]. When a state is entangled, subsystems A and B cannot be described independently with their state vectors, but instead the states of both are intrinsically linked.

To study a chosen subsystem independently, we must extract its local information from the overall state vector $|\psi\rangle$. We accomplish this with the *reduced density matrix*, defined for subsystem A as:

$$\rho_A = Tr_B(|\psi\rangle \langle\psi|) \quad (2)$$

Tr_B denotes the *partial trace* of subsystem B. For an orthonormal basis $\{|\mu\rangle\}_\mu$ of $\mathcal{H}_B$, we define the partial trace of subsystem B (from Debris-Alazard's Lecture Notes [8]) as:

$$Tr_B(|\psi\rangle \langle\psi|) = \sum_{\mu} (I \otimes \langle\mu|)(|\psi\rangle \langle\psi|)(I \otimes |\mu\rangle) \quad (3)$$

Physically, this 'traces out' all information of subsystem B from the overall density matrix, providing an isolated description of subsystem A. If $|\psi\rangle$ is an entangled state, then ρ_A will have more than one non-zero eigenvalue, and will thus be a mixed state, even if the original system was in a pure state. (by Chapter 2.4.1 of Preskill's Lecture Notes [9]).

The entanglement of a system can be quantified by the entanglement entropy, which measures how mixed a given reduced density matrix is. This is one of a plethora of measurement methods, and is best used when the overall system is a pure state. When measuring a mixed subsystem from an overall mixed system, it becomes difficult to separate the entanglement of the subsystem from the overall lack of purity of the full system.

Throughout this project, we will use two different measures, namely the von Neumann and Rényi-2 entropies. Given a reduced density matrix ρ_A with eigenvalues $\{\lambda_i\}$, these are:

Von Neumann Entropy:

$$S = -Tr(\rho_A \ln \rho_A) = -\sum_i \lambda_i \ln \lambda_i \quad (4)$$

Rényi-2 Entropy:

$$S_2 = -\ln Tr(\rho_A^2) = -\ln \sum_i \lambda_i^2 \quad (5)$$

Before continuing, we note one important property: for any arbitrary subsystem, $S_2 \leq S$ [10].

Although both measures are valid, the Rényi-2 entropy is generally easier to compute, as full eigendecomposition is not needed in its density matrix form. For this reason, the Rényi-2 entropy will serve as our primary form of measurement when studying the entanglement entropy of superpositions.

The entanglement entropy has a few other important properties:

- For product states, $S = S_2 = 0$.
- For any given subsystems A and B, $S(\rho_A) = S(\rho_B)$ and $S_2(\rho_A) = S_2(\rho_B)$ for pure states.
- The entanglement entropy for a subsystem A has a theoretical maximum of $S_{max} = N_A \ln(2)$. This occurs specifically when A has reduced density matrix $\rho_A = I/2^{N_A}$.
- Furthermore, consider a composite quantum system with disjoint subsystems A and B (with subsystem sizes $N_A \leq N_B$). Let $m = 2^{N_A}$ and $n = 2^{N_B}$, then by the paper from Page [11], there will be a difference between the maximum entropy and the average von Neumann single-state entropy of

$$I_{m,n} = \frac{m}{2n} + O\left(\frac{1}{mn}\right) \quad (6)$$

As $O(1/mn)$ approaches 0 when m,n approach infinity, the infinite limit of the mean von Neumann entropy should lie below S_{max} with a difference of $m/2n$.

We will use the first two properties as validation tests for our code in Sections 3.4.2 and 3.4.3, while the third and fourth properties will be discussed further in Section 4.

2.2 Quantum State Classes:

For this project, we consider three distinct families of quantum states.

2.2.1 Haar-Random States:

Haar-random states are uniformly random states sampled from the unit sphere in Hilbert space. To generate these, both the real and imaginary parts of each component of the state vector are sampled independently from the normal distribution $\mathcal{N}(0, 1)$. Normalizing the result will then give a Haar-random state.

Haar-random states have near-maximal entanglement for any bipartition, as we will see later. This makes them an interesting family of states to study, and a good comparison tool to the other state families considered.

2.2.2 Clifford Stabilizer States:

Clifford stabilizer states are constructed by applying sequences of Hadamard gates ($\hat{H}$), phase gates ($\hat{S}$) and CNOT gates to the vacuum state $|0\rangle^{\otimes N}$. The Hadamard and phase gates are single-qubit gates, and are defined as:

$$\hat{H} = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}, \quad \hat{S} = \begin{bmatrix} 1 & 0 \\ 0 & i \end{bmatrix} \quad (7)$$

The CNOT gate instead acts on 2 qubits within the overall system, a target qubit and a control qubit. Its action is defined as follows: If the control qubit is $|0\rangle$, it does nothing to the target qubit, and if the control qubit is $|1\rangle$, it flips the target qubit. This does not have a fixed matrix form, as its matrix form depends on the location of the target and control qubits.

By the Gottesman-Knill Theorem, Clifford stabilizer states can be simulated in polynomial time on a classical computer [12]. However, they are insufficient on their own to make a universal quantum computer, without the addition of non-Clifford gates (with the typical addition being the T gate). Despite their ability to be simulated classically, Clifford stabilizer states have near-maximal entanglement, similarly to Haar-random states.

2.2.3 Number-Conserving States:

Free-fermion/Gaussian states are a special type of state which describe systems of non-interacting fermions. Number-conserving states are a subclass of free-fermionic states which describe systems with a fixed particle number. Since these states obey the Pauli Exclusion Principle, they are described using Slater determinants (an anti-symmetrized product of single-fermion wavefunctions).

To construct these, we begin by considering the *spin operators*, defined as:

$$\sigma^- = \begin{bmatrix} 0 & 0 \\ 1 & 0 \end{bmatrix}, \quad \sigma^+ = \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix} \quad (8)$$

These matrices act to raise or lower the spin of a single-qubit state; observe that $\sigma^+ |0\rangle = |1\rangle$ and $\sigma^- |1\rangle = |0\rangle$. We can extend this creation/destruction of fermions to systems of multiple qubits by applying the *Jordan-Wigner transformation*. To accomplish this, we define the fermionic creation and annihilation operators $c_i^\dagger$ and c_i respectively. These are given by the following formulae (also known as the *Jordan-Wigner strings*):

$$c_i = \left[\prod_{j=1}^{i-1} \sigma_j^z \right] \sigma_i^-, \quad c_i^\dagger = \left[\prod_{j=1}^{i-1} \sigma_j^z \right] \sigma_i^+ \quad (9)$$

Note σ_j^z denotes the matrix

$$\sigma_z = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix} \quad (10)$$

applied to the j -th qubit. Intuitively, c_i annihilates a fermion at site i , while $c_i^\dagger$ creates a fermion at site i . The product of σ_z operators is included to encode the antisymmetry mentioned above and to ensure the Jordan-Wigner strings satisfy the canonical anti-commutation relations:

$$\{c_i, c_j\} = 0, \quad \{c_i^\dagger, c_j^\dagger\} = 0, \quad \{c_i, c_j^\dagger\} = \delta_{ij} I \quad (11)$$

However, these operators by themselves do not allow for the mixture of creation and annihilation operators, which restricts us from accessing superpositions of states. This is where we introduce *Bogoliubov transformations*, which are explicitly constructed as a sum of creation and/or annihilation operators to circumvent this. For number-conserving operators, we only use creation operators, and construct Bogoliubov operators by the following sum:

$$\beta_n = \sum_x U_{n,x} c_x^\dagger \quad (12)$$

Here, U is a random unitary matrix of size $N \times N$, which we will sample n columns from. These column vectors define the single-particle wavefunctions for the Slater determinant. To generate a number-conserving state with n fermions, we apply n Bogoliubov operators to the vacuum state $|0\rangle^{\otimes N}$, i.e.

$$|\psi\rangle = \beta_1 \beta_2 \dots \beta_n |0\rangle^{\otimes N} \quad (13)$$

Number-conserving states have *sub-maximal entanglement*, lying well below the maximum entanglement entropy S_{max} . Since particle number is conserved, a system described by a number-conserving state vector can only be bipartitioned into configurations which preserve total particle number, limiting the maximum entanglement entropy. This makes them an interesting contrast to the Haar-random and Clifford families of states.

2.3 Page Curves & Equal-Weight Superpositions:

Page curves are an important tool for characterizing entanglement entropy with respect to subsystem size. For a system of N qubits, the Page curve plots the average entanglement entropy $\langle S \rangle$ against each possible subsystem size $N_A \in \{1, \dots, N-1\}$. Different state classes will have different Page curves, which allows us to compare the average behaviour of the entanglement entropies of each. We expect both Haar-random and Clifford stabilizer states to have Page curves only slightly below $S_{max} = N_A \ln(2)$, while number-conserving states should have their Page curves lie significantly below this maximum.

For any arbitrary set of m independent random states $\{|\psi_i\rangle : i = 1, \dots, m\}$, we define their equal-weight superposition as follows:

$$|\Psi_m\rangle = \frac{1}{\mathcal{N}} \sum_{i=1}^m |\psi_i\rangle \quad (14)$$

$\mathcal{N}$ is a normalization factor for the superposition. Since these states are not necessarily orthogonal, $\mathcal{N} \neq \sqrt{m}$ in general. Let $S_2(m)$ denote the Rényi-2 entropy of this equal-weight superposition.

To study the effect of the equal-weight superposition on the entropies of its states, we introduce the measure $\Delta S_2(m)$. For a collection of m independent random states, this will measure the difference between the mean single-state entropy and the entropy of their equal weight superposition:

$$\Delta S_2(m) = S_2(m) - \langle S_2(1) \rangle \quad (15)$$

Here, $S_2(1)$ is the mean Rényi-2 entropy of each of the individual components:

$$S_2(1) = \frac{1}{m} \sum_{i=1}^m S_2(|\psi_i\rangle) \quad (16)$$

If $\Delta S_2 > 0$, then the equal-weight superposition of our set of states has a higher entanglement entropy than its average component. Likewise, if $\Delta S_2(m) < 0$, this superposition has less entanglement entropy than its average component.

This leads us into the central question of this project: how does ΔS_2 behave for Haar-random, Clifford stabilizer and number-conserving states? Specifically, how is ΔS_2 affected for each state class as we increase the number of states m , and does this reveal anything about the relationship between the structure of states and the superposition entanglement entropy? We will attempt to answer these questions below.

3 Implementations:

3.1 Reduced Density Matrix Calculator:

As discussed in Section 2.1, studying individual subsystems requires us to find the reduced density matrix of the subsystem by taking a partial trace. For this partial trace, we avoid constructing the density matrix ρ of the full system. For a system of N qubits, this is a $2^N \times 2^N$ matrix, and for $N = 16$, this would require storing $2^{16} \times 2^{16} = 4,294,967,296$ values, clearly computationally infeasible.

Instead, we implement the method of Qiskit's `partial_trace` function [13]. To begin, we reshape our state vector of size 2^N into an N -dimensional tensor T with shape $[2, 2, \dots, 2]$, where each index corresponds to a single qubit. For instance, for $N = 3$, $T[0, 1, 0]$ would give the amplitude of the basis state $|010\rangle$. Next, since the computational basis states $\{|i_1 \dots i_N\rangle : i_1, \dots, i_N \in \{0, 1\}\}$ are an orthonormal complete basis, applying equation (3) from Section 2.1 will give the entries of the reduced density matrix as:

$$\rho_A[i, i'] = \sum_{j_1, \dots, j_{N_B}} T[i, j_1, \dots, j_{N_B}] T^\dagger[i', j_1, \dots, j_{N_B}] \quad (17)$$

where $j_1, \dots, j_{N_B}$ are the qubits to be traced out, and i, i' are the row and column indices of an entry in ρ_A . NumPy's `tensordot` function is used to sum over all configurations of the traced out qubits, giving the values of the reduced density matrix ρ_A in a tensor of shape $[2, \dots, 2]$ with $2N_A$ indices (with each term corresponding to a particular combination of row and column indices of ρ_A). Finally, we reshape the results back into a matrix of size

$2^{N_A} \times 2^{N_A}$. This method avoids the computational cost of generating the full density matrix of the system, and works efficiently for systems of up to 16 qubits.

3.2 Measuring the Entanglement Entropy:

With this partial trace calculator in place, we now build functions to calculate the von Neumann and Rényi-2 entanglement entropies from a given density matrix. For the von Neumann entropy, we use its eigenvalue form

$$S = - \sum_i \lambda_i \ln \lambda_i \quad (18)$$

where λ_i are the eigenvalues of the density matrix ρ_A (ρ_A is calculated using the reduced density matrix calculator from Section 3.1). Since all density matrices are Hermitian, we find these eigenvalues using NumPy's `eigvalsh` calculator, which exploits this fact for a faster algorithm than the regular `eigvals` calculator. As a safety check, we exclude any zero eigenvalues with a tolerance of 10^{-12} to prevent any potential errors from $\ln(0)$ being undefined.

The Rényi-2 entropy is calculated directly from the reduced density matrix:

$$S_2 = - \ln \text{Tr}(\rho_A^2) \quad (19)$$

This is the more efficient calculation of the Rényi-2 entropy compared to its eigenvalue form, since, as discussed before, no eigendecomposition is needed.

3.3 State Generators:

We proceed to make functions to randomly sample each of the three state families outlined in Section 2.2. For each state generator, we use NumPy's `random.default_rng` function to generate random numbers. This allows us to sample randomly while maintaining reproducibility (by feeding this function the same seed). This is important for reproducing graphs and for preventing any potential 'bad behaviour' from unlucky random number generation.

3.3.1 Haar-Random States:

To construct these, we generate 2^N numbers, randomly sampling both the real and imaginary parts of each value from $\mathcal{N}(0,1)$. We then place these into a NumPy array, and normalise the resulting vector. We repeat this for however many states are desired.

Note: For this generator, we opted not to include a sparse implementation. Analytically, Haar-random states will never have a zero entry, because as the normal distribution $\mathcal{N}(0, 1)$ is continuous,

$$P(X = 0) = \int_0^0 f(x)dx = 0 \quad (20)$$

where $f(x)$ is the probability density function of $\mathcal{N}(0, 1)$. In practice, we may still get values equal to zero due to computational rounding, but almost all entries of a Haar-random state will have non-zero values, so a sparse implementation would provide minimal memory improvements.

3.3.2 Clifford Stabilizer States:

From the paper by Aaronson and Gottesman [14], the number of distinct N-qubit Clifford stabilizer states is given by:

$$2^N \prod_{i=0}^{N-1} (2^{N-i} + 1) \quad (21)$$

This makes generating the full set of Clifford stabilizer states impractical for larger system sizes due to memory costs, so we instead choose to randomly sample them. Building an adequate generator to achieve this proved to be more difficult than originally thought, requiring three separate iterations.

Our first approach was to generate a random number in a certain number range (typically [500, 1000]) and apply that many random Hadamard, phase, and CNOT gates to the vacuum state $|0\rangle^{\otimes N}$. This process would be repeated for every state with a new random number each time. This was done using Stim’s `TableauSimulator` environment for efficiency. However, we quickly realised that using such a sample was unreliable, as certain random walks of operators would be favoured by the generator. Specifically, any operations requiring fewer gates to reach are always more likely, creating an inherent bias in our sample.

The second iteration availed of Stim’s `Tableau.Random` function, which generates a random Clifford operation to apply onto the `TableauSimulator` object. However, it turns out that states generated in the Stim environment are canonicalised such that zero values are always exactly 0, and that the first non-zero entry is positive, which always gives the states generated the same phase. This was a substantial factor as to why our first two approaches were not accurate, and led to unphysical behaviour, such as the Clifford superposition entropy $S_2(m)$ decreasing well below the Haar-random mean single-state entropy $\langle S_2(1) \rangle$.

The final implementation is instead built with Qiskit, namely the function `random_clifford` from Qiskit’s `quantum_info` package, which samples uniformly from the set of all Clifford operations (each element is a random composition of multiple Hadamard, phase and CNOT gates). For each state generated, we use this to sample one operation, convert it into a Qiskit circuit to prevent constructing the full matrix, and apply this to the Statevector of

the vacuum state. We then convert the output object into a NumPy array, and repeat as many times as necessary.

3.3.3 Number-Conserving States:

To generate this type of state, we first need a method to generate a random unitary matrix. To do this, we use Mezzadri's method [15]:

1. First, we create a matrix M of size (N, N) by taking an independent, identically distributed sample from the complex Gaussian distribution (i.e. we take Gaussian distributions for both the real and imaginary parts of each value of the matrix.)
2. Next, we take the QR decomposition of the matrix. To accomplish this, we rewrite $M = QR$, where Q is a unitary matrix, and R is an upper-triangular, invertible matrix. This can be done using Scipy's `linalg.qr` function.
3. To ensure true randomness with respect to the Haar measure, we construct the diagonal matrix

$$\Lambda = \begin{bmatrix} \frac{r_{11}}{|r_{11}|} & & & \\ & \frac{r_{22}}{|r_{22}|} & & \\ & & \ddots & \\ & & & \frac{r_{nn}}{|r_{nn}|} \end{bmatrix} \quad (22)$$

where the entries r_{ii} are the diagonal entries of R , and $|r_{ii}|$ is the absolute value of each entry.

4. Finally, the matrix $Q' = Q\Lambda$ will be a random unitary matrix distributed with respect to the Haar measure.

Using this method, we construct an $N \times N$ unitary matrix and take its first n columns as single-particle wavefunctions to construct our Bogoliubov operators via formula (12). A few key things to note:

- When calculating our Jordan-Wigner strings for our function, we pre-calculate all values of $c_i^\dagger$ for $i \in \{1, \dots, N\}$. This is done purely for optimization purposes; it will prevent us from repeatedly finding these values for every state generated.
- For a similar reason, each of these operators is computed and stored as a sparse matrix. Each one of these operators is the tensor product of σ_z and either σ^+ or σ^- , and will only act non-trivially on a few qubits of a given state. As such, most values of these operators will be zero, meaning sparse matrices will substantially alleviate computational cost.

Each Bogoliubov operator will also be a sparse matrix, as they are a linear combination of the sparse Jordan-Wigner strings. We then apply these to the state $|0\rangle^{\otimes N}$. This is a dense matrix for easy compatibility with other functions, but SciPy automatically handles the sparse-dense product, returning a dense vector. The resulting state is then normalized. Doing this allows us to obtain a number-conserving state with n fermions. We repeat this until we reach the desired number of states, generating a new random unitary matrix each time.

Note: Throughout this project, for a system of N qubits, we will take number-conserving states with $n := N/2$ rounded **down**, unless specified otherwise.

3.4 Validation Tests:

We now carry out a few validation tests to ensure our above reduced density matrix and entropy calculators are working correctly. We test three commonly known properties of the entanglement entropy to verify this.

3.4.1 Bell State Test:

The *Bell states* are a well-documented set of 2-qubit states, and provide a useful test case to corroborate our partial trace and entropy calculation methods. They are defined as:

$$\begin{aligned} |\beta_{00}\rangle &= \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle) & |\beta_{01}\rangle &= \frac{1}{\sqrt{2}}(|01\rangle + |10\rangle) \\ |\beta_{10}\rangle &= \frac{1}{\sqrt{2}}(|00\rangle - |11\rangle) & |\beta_{11}\rangle &= \frac{1}{\sqrt{2}}(|01\rangle - |10\rangle) \end{aligned} \quad (23)$$

These four states have an interesting property; for any single-qubit subsystem, they have maximal entanglement $S_{max} = \ln(2)$.

To validate this, we computed the reduced density matrices of both single-qubit subsystems of a 2-qubit quantum system, and then computed both the von Neumann and Rényi-2 entropies for each subsystem. From our testing, each state gave the exact same values for both single-qubit subsystems, namely the following:

	Value:	Absolute Error:
Maximum Entanglement Entropy:	0.6931471805599453	0
von Neumann Entropy:	0.6931471805599454	$\sim 10^{-16}$
Rényi-2 Entropy:	0.6931471805599457	$\sim 4 \times 10^{-16}$

Table 1: Entropy values for each Bell State, for both single-qubit subsystems.

As we can see, both measures of the entanglement entropy agree with the theoretical maximum to within $\sim 10^{-16}$, confirming the accuracy of our implementation.

Interestingly, each Bell state shares the same reduced density matrix:

$$\rho_A = \frac{I}{2} = \begin{bmatrix} 1/2 & 0 \\ 0 & 1/2 \end{bmatrix} \quad (24)$$

This corresponds to the reduced density matrix of a maximally-mixed state of a single-qubit subsystem (which has general form $\rho_A = I/2^{N_A}$ for a subsystem with N_A qubits). It is also consistent with the result from Section 2.1 that entangled pure states can have mixed reduced density matrices.

3.4.2 Product State Test:

We now proceed to test the entropy for a selection of product states. As stated in Section 2.1, any product state should have zero entanglement entropy. We test the entropy of all 2^N computational basis states $|0\dots 0\rangle, |0\dots 1\rangle, \dots, |1\dots 1\rangle$, for $N \in \{1, \dots, 6\}$. These are all trivially product states, as they can always be written as the tensor product of 1-qubit states. We test these computational basis states for all possible bipartitions of the system.

For all states tested, both the von Neumann and Rényi-2 entropies returned values of zero for all system sizes. This is because any computational basis state in the Fock basis will have a single eigenvalue $\lambda = 1$, and all other eigenvalues equal to 0. Hence, we get the entropies

$$S = - \sum_i \lambda_i \ln \lambda_i = -1 \ln(1) = 0 \quad (25)$$

$$S_2 = - \ln \sum_i \lambda_i^2 = - \ln(1) = 0 \quad (26)$$

However, this test alone doesn't conclusively prove that all product states have zero entropy. As an extra precaution, we added an additional test `bonus_prod_test` in our code. This test verifies that the states $|\psi_1\rangle = \frac{1}{\sqrt{2}}(|00\rangle + |01\rangle)$ and $|\psi_2\rangle = \frac{1}{2}(|00\rangle + |01\rangle + |10\rangle + |11\rangle)$ have entanglement entropies of zero according to both measures. These are both product states as they have a Schmidt number of one, yet lie outside the set of computational basis states. Although testing the set of all product states is impossible (as there are infinitely many product states), these two tests in combination provide reasonable confirmation of our above methodologies.

3.4.3 Subsystem Test:

Finally, we verify that for any bipartition of subsystems A and B, $S(\rho_A) = S(\rho_B)$ and $S_2(\rho_A) = S_2(\rho_B)$. Here, we took a system of $N = 4$ qubits, splitting into subsystems A of qubits 1 and 3, and B of qubits 2 and 4. We tested 100 randomly-generated states of each of the Haar-random, Clifford and number-conserving states using their generators from Section 3.3, and compared both. Both measures agreed to all states consistently up to $\sim 10^{-12}$, with only occasional errors at $\sim 10^{-15}$, likely due to rounding errors.

This is a particularly important property to verify, as it will later become essential for our Page curve plots.

3.5 Page Curve Plots:

We now discuss the method of plotting Page curves. For a system of N qubits, we start with a range of subsystem sizes $N_A \in \{1, \dots, N-1\}$ and a sample of states generated from any of our generators in Section 3.3. We proceed as follows:

1. For a particular system size N_A , we compute the reduced density matrix of the first N_A qubits of our system. We do this for every state in the sample. Since we average

over a large amount of states, the choice of which qubits lie in our subsystem should not substantially affect the results, as long as we take the same subsystem sizes.

2. Both the von Neumann and Rényi-2 entropies are calculated for each reduced density matrix.
3. We calculate the mean and the standard deviation of each entropy across the whole state sample. This returns the Page curve value for each entropy for a single value of N_A .
4. We repeat this across all values of N_A , giving us a set of values of the mean entanglement entropy to plot against system size.

A key optimisation is made here. Recall from Sections 2.1 and 3.4 that for any bipartition of a system into subsystems A and B , $S(\rho_A) = S(\rho_B)$. Using this, we add a check to always take the entanglement entropy of the smaller subsystem. Specifically, if $N_A > N/2$, we instead compute the reduced density matrix of the last $N - N_A$ qubits. This avoids the explicit calculation of reduced density matrices for larger subsystems, and limits the size of any considered matrix to a maximum of $2^8 \times 2^8$ for calculations of up to 16 qubits. This allows larger subsystems to be explored efficiently while producing identical results.

To study finite-size effects, we plot the Page curves for system sizes $N = 6, 8, 12, 16$. Additionally, we record the values at the peak of each Page curve, at $N_A = N/2$. Only even subsystem sizes will be considered, as $N_A = N/2$ is not an integer if N is odd. The Page curve plots for each state family, as well as comparison plots of all 3 families, are presented in Section 4.1.

3.6 Superposition Analysis Plots:

We now move into the core investigation of this project. Suppose we take a given set of values $m \in \{1, \dots, m_{max}\}$, representing the number of states included in the equal-weight superposition. We describe the procedure to plot ΔS_2 (as defined in equation (15)) against the number of states m for a given state family. For each of a fixed number of trials, we generate a set of m_{max} states with the relevant state generator from Section 3.3. Within each trial, we do the following:

1. For each value of m , we take the first m states from our sample.
2. Using these states, we construct the equal-weight superposition from equation (15), and calculate the equal-weight superposition Rényi-2 entropy $S_2(m)$. Note that the normalization factor $\mathcal{N}$ is taken as the Euclidean norm from NumPy's `linalg.norm` function.
3. Additionally, we calculate the Rényi-2 entropy of each individual state in our selection, and take the resulting average. This gives $\langle S_2(1) \rangle$ for a single m value for a single trial. (Note that for both quantities, when finding our reduced density matrix, we trace out the last $N/2$ qubits unless specified otherwise.)

4. We finally find $\Delta S_2(m)$ by taking their difference (as seen in equation (14)).
5. We repeat this for every value of m , giving us a list of values for each of the three quantities for $m \in \{1, \dots, m_{max}\}$ per trial.

We repeat this process until we reach the desired number of trials. Finally, using the data for all trials, we calculate the mean and standard deviations of $S_2(m)$, ΔS_2 , $\langle S_2(1) \rangle$. This gives us a set of values for all 3 quantities which we plot against the values of m . By using the same process as above, we also implement functions which compare these quantities for different state classes, and for different system and subsystem sizes.

The plots of each state family, as well as comparison plots of all 3 families, can be seen below in Section 4.2.

4 Results:

4.1 Page Curve Analysis:

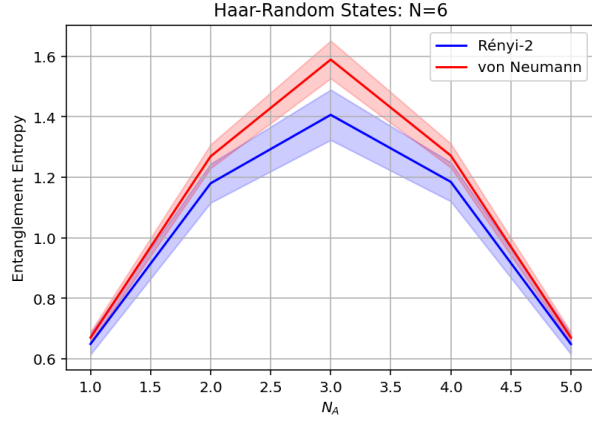
We now present the Page Curve plots obtained for each of the Haar-random, Clifford stabilizer and number-conserving states, generated using the process of Section 3.5. Throughout all plots below, the standard deviation region $(-\sigma, +\sigma)$ is denoted by a shaded region around the mean.

4.1.1 Haar-Random States:

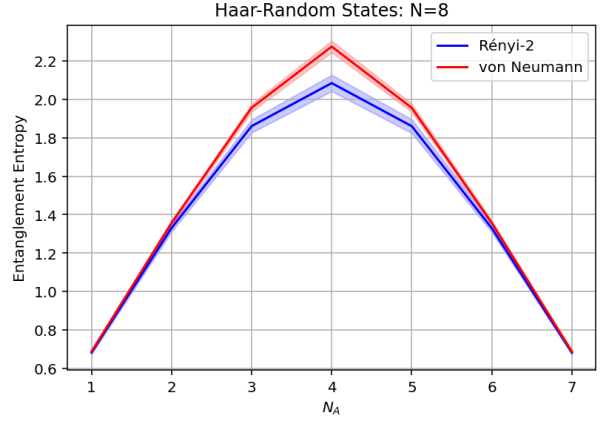
In Figure 1, we plot the Haar-random Page curves for $N = 6, 8, 12, 16$, each for a sample of 1000 randomly-generated states. In Table 2, we record the mean and standard deviations at $N_A = N/2$ for both entropies for each plot.

N	$\langle S_2 \rangle$	$\langle S \rangle$	Ratio ($\langle S_2 \rangle / \langle S \rangle$)	σ_{S_2}	σ_S	Ratio (σ_{S_2} / σ_S)
6	1.4064	1.5892	~ 0.885	0.0832	0.0622	~ 1.337
8	2.0848	2.2749	~ 0.9164	0.0424	0.0303	~ 1.3993
12	3.4657	3.6587	~ 0.9472	0.0108	0.0077	~ 1.4024
16	4.8519	5.045	~ 0.9617	0.0029	0.002	~ 1.4181

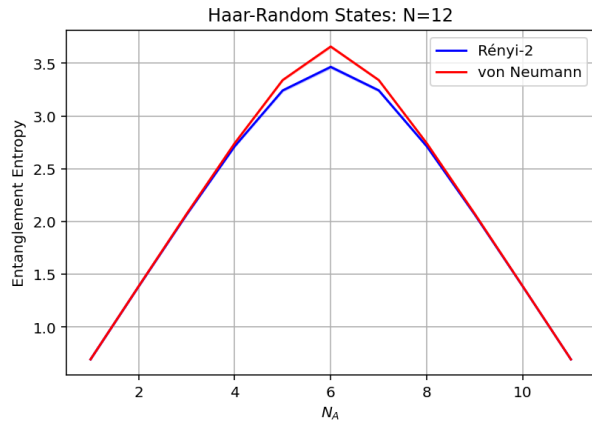
Table 2: Mean and standard deviation values at $N_A = N/2$ for each plot in Figure 1.



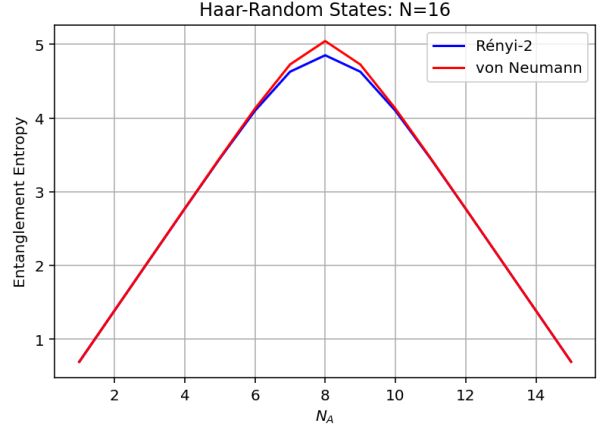
(a) The entanglement entropies of 1000 Haar-random states for a 6-qubit system.



(b) The entanglement entropies of 1000 Haar-random states for a 8-qubit system.



(c) The entanglement entropies of 1000 Haar-random states for a 12-qubit system.



(d) The entanglement entropies of 1000 Haar-random states for a 16-qubit system.

Figure 1: Haar-random Page curves for different system sizes.

From Figure 1, for each value of N , the mean entanglement entropies of both measures reach their maximum at $N_A = N/2$, increasing monotonically up to and decreasing monotonically away from this value. The Page curve is symmetric around this maximum for all system sizes. This is consistent with our earlier property from Section 2.1, namely that for a bipartition of subsystems A and B , $S(\rho_A) = S(\rho_B)$. We also observe $\langle S_2 \rangle \leq \langle S \rangle$, again in agreement with the theoretical result from Section 2.1.

From Table 2, we observe that the ratio $\langle S_2 \rangle / \langle S \rangle$ increases monotonically with system size. However, the difference of $\langle S_2 \rangle - \langle S \rangle$ seems to converge to some constant value, with differences 0.1828, 0.1901, 0.193 and 0.1931 for $N = 6, 8, 12, 16$ qubits respectively. Thus, the Rényi-2 and von Neumann mean entropies of Haar-random states appear to share a common rate of increase (a result to be seen again in Section 4.1.4).

The standard deviations of both entropies decrease significantly as we increase the system

size, so much so that the standard deviation bands become completely unobservable for $N = 16$. In particular, from Table 2, the standard deviations decrease by a factor of 28 for the Rényi-2 entropy, and by a factor of 31 for the von Neumann entropy, between systems of 6 and 16 qubits. This suggests that Haar-random states become increasingly concentrated around their mean values as we increase the system size.

Finally, the ratio between the Rényi-2 and von Neumann standard deviations remains roughly constant for sufficiently large system sizes, with an approximate ratio $\sigma_{S_2}/\sigma_S \approx 1.4$. This suggests that the fluctuations of both entropy measures scale proportionally to each other as we increase system size.

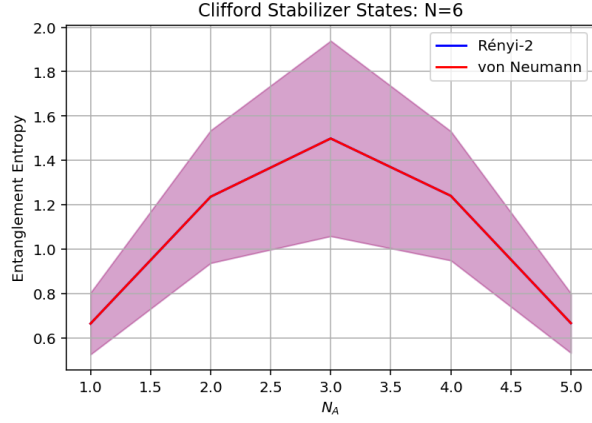
4.1.2 Clifford Stabilizer States:

In Figure 2, we plot the Page curves for $N = 6, 8, 12, 16$ for a sample of 1000 randomly-generated Clifford stabilizer states. In Table 3, we record the mean and standard deviation of both entropies at $N_A = N/2$ for each value of N .

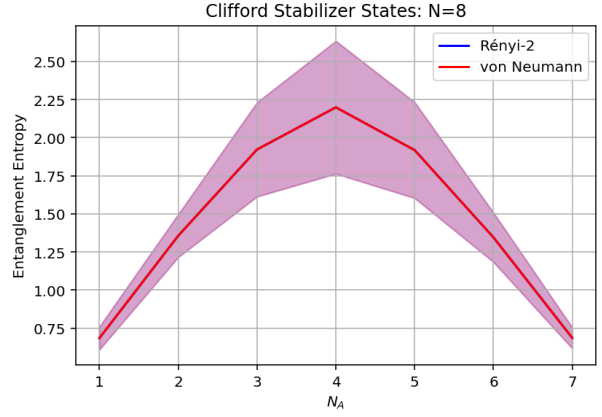
From Figure 2, the Clifford Page curve shares several similarities with its Haar-random counterpart. The Clifford Page curve has its maximum entanglement entropies at $N_A = N/2$, and is symmetric around this point. The overall shape of the Clifford Page curve is also similar to that of Haar-random states, increasing monotonically to and decreasing monotonically from the maximum.

However, the Clifford stabilizer states show one striking difference; in both Figure 2 and Table 3, we observe that the mean and standard deviations of both entropy measures match exactly for all system sizes. Furthermore, by testing the Page curves of individual states, we confirm S and S_2 have identical means and deviations for each individual Clifford stabilizer state, up to $\sim 10^{-13}$ (this is illustrated in Figure 3 below). This is a remarkable structural property of the Clifford stabilizer states.

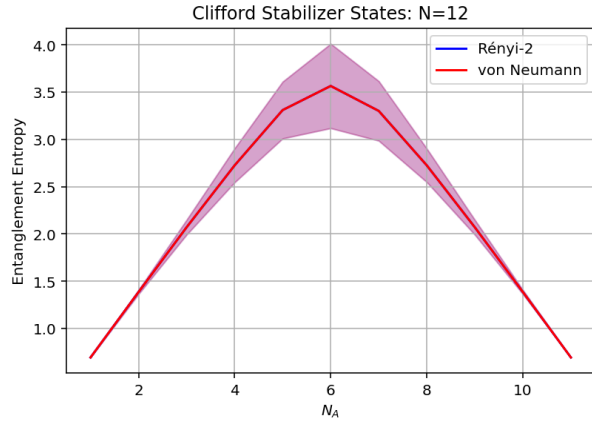
The standard deviations of the Clifford entropies also differ from those of Haar-random states in several ways. Firstly, they are not significantly affected by system size, with an approximate value $\sigma \sim 0.44$ at $N_A = N/2$ for each value of N . These values are much larger than the Haar-random standard deviations, illustrating the higher variance of Clifford states. Additionally, the Clifford standard deviation is noticeably larger at maximum entropy values than it is for the smaller and larger subsystem sizes for each plot. This suggests that the entropies of Clifford stabilizer states do not concentrate around the mean entropy in the same way Haar-random states do.



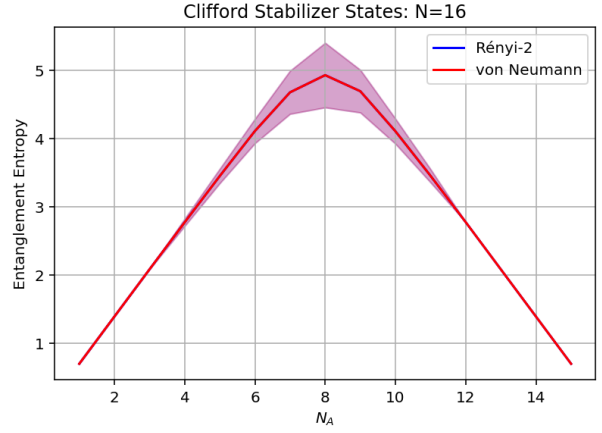
(a) The entanglement entropies of 1000 Clifford stabilizer states for a 6-qubit system.



(b) The entanglement entropies of 1000 Clifford stabilizer states for a 8-qubit system.



(c) The entanglement entropies of 1000 Clifford stabilizer states for a 12-qubit system.



(d) The entanglement entropies of 1000 Clifford stabilizer states for a 16-qubit system.

Figure 2: Clifford stabilizer state Page curves for different system sizes.

N	$\langle S_2 \rangle$	$\langle S \rangle$	Ratio ($\langle S_2 \rangle / \langle S \rangle$)	σ_{S_2}	σ_S	Ratio (σ_{S_2} / σ_S)
6	1.4986	1.4986	1	0.4404	0.4404	1
8	2.1994	2.1993	1	0.4357	0.4357	1
12	3.5655	3.5655	1	0.4467	0.4467	1
16	4.9566	4.9566	1	0.4588	0.4588	1

Table 3: Mean and standard deviation values at $N_A = N/2$ for each plot in Figure 2.

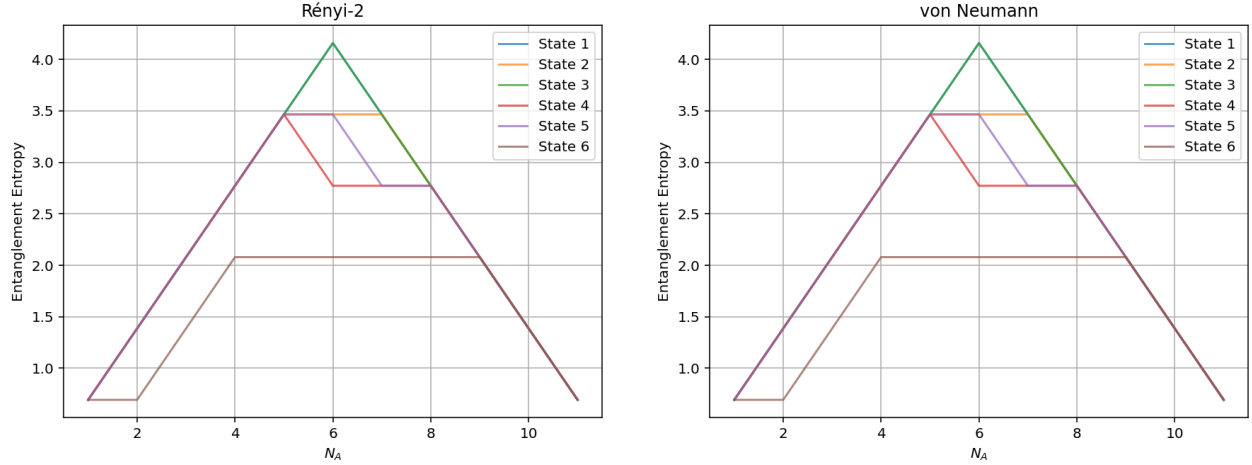


Figure 3: The individual entropies of 6 random 12-qubit Clifford stabilizer states, compared for both the von Neumann and Rényi-2 entropies.

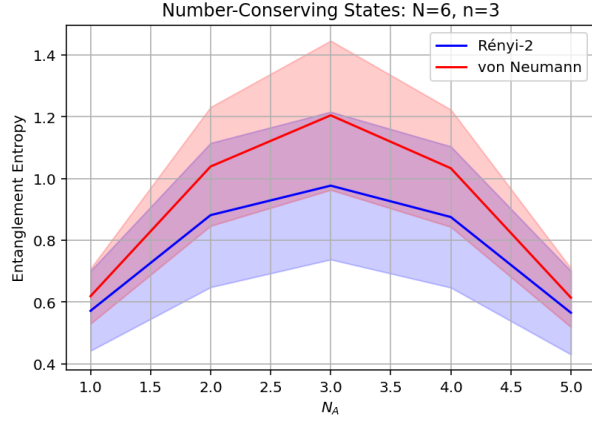
4.1.3 Number-Conserving States:

In Figure 4, we plot the Page curves for $N = 6, 8, 12, 16$ for a sample of 1000 randomly-generated number-conserving states. In Table 4, we record the corresponding mean and standard deviation of both entropies at $N_A = N/2$ for each value of N .

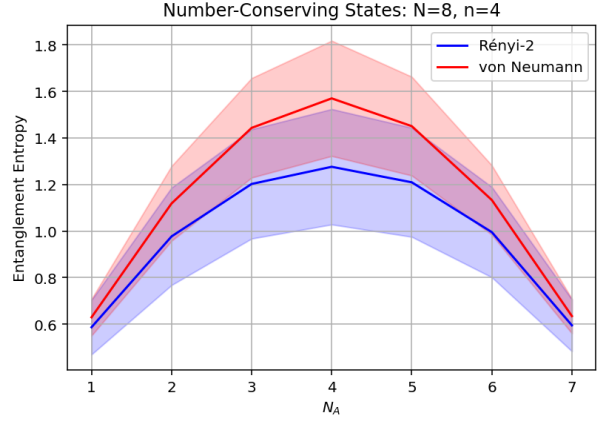
Number-conserving states present the largest contrast compared to the other two state families. From Figure 4, we again see a symmetric Page curve, with its maximum at $N_A = N/2$. However, we observe that the Page curve has a more rounded peak than either the Haar-random or Clifford stabilizer Page curves, and a lower maximum value for the Rényi-2 and von Neumann entropies for all values of N . This confirms the sub-maximal entanglement of number-conserving states, and will be explored further via direct comparison of all state families in Section 4.1.4.

Like Haar-random states, number-conserving states show a clear distinction between the Rényi-2 and von Neumann entropies, with $\langle S_2 \rangle \leq \langle S \rangle$ for all system sizes. The ratio $\langle S_2 \rangle / \langle S \rangle$ is near-constant with an approximate value of 0.81, only increasing by ~ 0.01 at the peak of its Page curve between systems of 6 and 16 qubits. This is an intriguing result, and indicates that the von Neumann and Rényi-2 entropies increase proportionally to each other for number-conserving states as we increase system size.

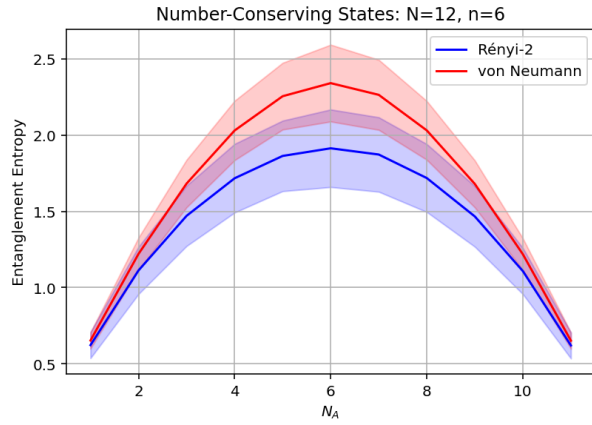
The standard deviations of both the Rényi-2 and von Neumann entropies are approximately equal, with an approximately constant ratio $\sigma_{S_2} / \sigma_S \approx 1$ by Table 4. Similarly as for Clifford stabilizer states, the standard deviations of number-conserving Page curves are not noticeably affected by system size.



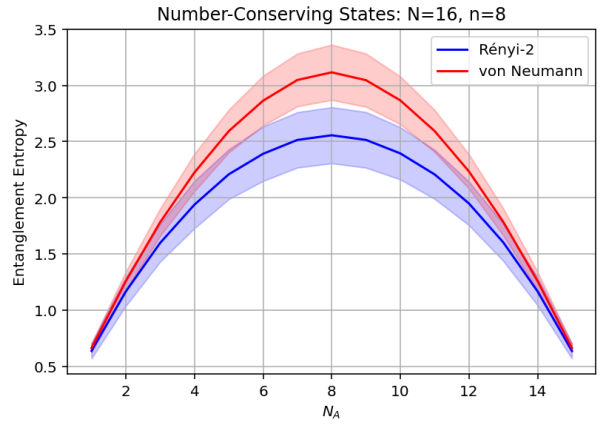
(a) The entanglement entropies of 1000 number-conserving states with 3 fermions for a 6-qubit system.



(b) The entanglement entropies of 1000 number-conserving states with 4 fermions for a 8-qubit system.



(c) The entanglement entropies of 1000 number-conserving states with 6 fermions for a 12-qubit system.



(d) The entanglement entropies of 1000 number-conserving states with 8 fermions for a 16-qubit system.

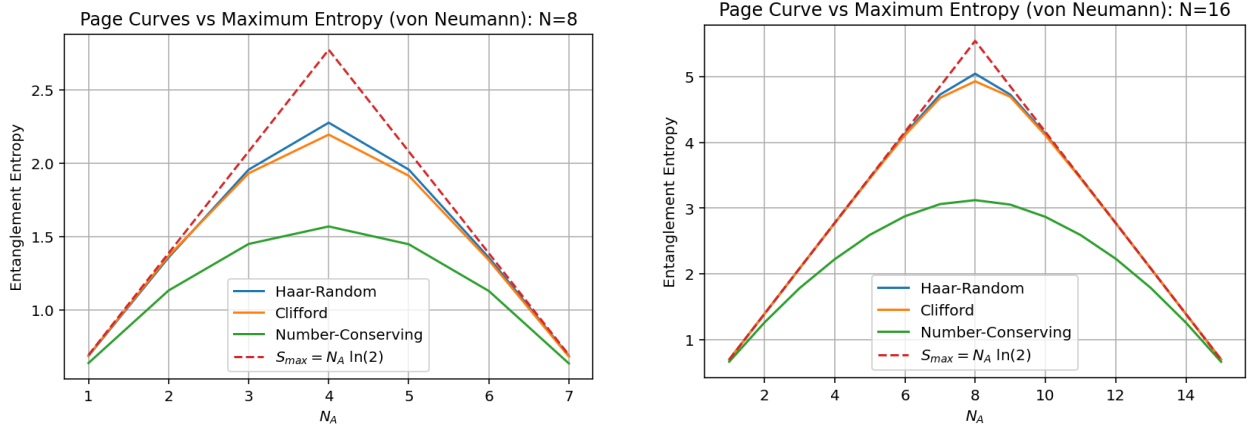
Figure 4: Number-conserving Page curves for different system sizes.

N	$\langle S_2 \rangle$	$\langle S \rangle$	Ratio ($\langle S_2 \rangle / \langle S \rangle$)	σ_{S_2}	σ_S	Ratio (σ_{S_2} / σ_S)
6	0.9767	1.2044	~ 0.811	0.2392	0.2415	~ 0.9902
8	1.2764	1.5704	~ 0.8128	0.2479	0.2476	~ 1.0011
12	1.914	2.3416	~ 0.8174	0.2545	0.2519	~ 1.0103
16	2.5562	3.1165	~ 0.8202	0.2498	0.246	~ 1.0155

Table 4: Mean and standard deviation values at $N_A = N/2$ for each plot in Figure 4.

4.1.4 Comparisons & The Theoretical Maximum:

In Figure 5, we plot all 3 of our Page curves for the von Neumann entropy for system sizes $N = 8, 16$ and compare them to the theoretical maximum $S_{max} = N_A \ln(2)$.



(a) The Page Curves of 500 of each state family plotted against $S_{max} = N_A \ln(2)$ for $N = 8$ qubits.

(b) The Page Curves of 500 of each state family plotted against $S_{max} = N_A \ln(2)$ for $N = 16$ qubits.

Figure 5: Comparison of the von Neumann entropies for 500 states of each state family against the theoretical maximum $S = N_A \ln(2)$.

For $N = 8$, the Haar-random and Clifford mean entropies are close, but share a small discrepancy, with the Clifford Page curve lying slightly below the Haar Page curve. Both Page curves also lie significantly below the theoretical maximum.

Meanwhile, for $N = 16$, the Haar-random and Clifford stabilizer mean entropies are nearly indistinguishable. For instance, from Table 2, the Haar-random von Neumann peak is 5.045 for $N = 16$, approximately 91% of the maximum $S_{max} = 8 \ln(2)$, compared to $2.2749/(4 \ln(2)) \approx 82\%$ at $N = 8$. Thus, we see that these entropies more closely approach the maximum entropy as we increase system size. Yet, $8 \ln(2) - 5.0452 \approx 0.5$, and $4 \ln(2) - 2.2749 \approx 0.5$, both consistent with a difference of $1/2$ between the maximum entropy and the measured von Neumann entropy. This agrees with Page's original paper, as examined in Section 2.1.

By contrast, the number-conserving Page curve lies significantly below the maximum entropy and the other two Page curves. The difference between the number-conserving mean entropy and its Haar-random and Clifford counterparts is the largest for $N_A = N/2$, and decreases monotonically as we increase or decrease the subsystem size away from $N_A = N/2$.

We illustrate these trends further in Figure 6, where we compare the 'peaks' of each Page curve for $N = 4, 6, 8, 10$ (i.e. taking the first $N/2$ qubits for each Page curve in a system of N qubits). We observe that the peaks of the Page curves of all 3 state families increase linearly with respect to system size. The Haar-random and Clifford Page curves increase at

an approximately proportional rate to S_{max} , while the number-conserving Page curve scales at a substantially lower rate.

One other thing we notice is that the Rényi-2 entropy is greater for Clifford stabilizer states than Haar-random states, while the opposite is true for the von Neumann entropy. This is a consequence of $\langle S_2 \rangle$ and $\langle S \rangle$ being equal for the Cliffords, while having a constant difference for Haar-random states.

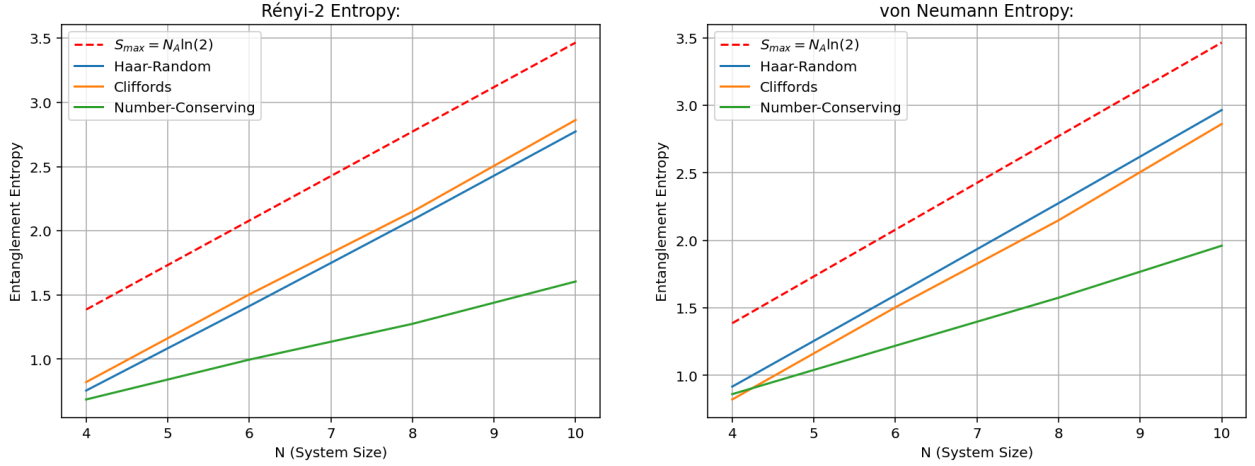


Figure 6: The maximum von Neumann and Rényi-2 entropies, plotted for $N \in \{4, 6, 8, 10\}$ (sampling 100 states per family).

4.2 Superposition Analysis:

As discussed in Section 3.6, we now plot the quantities $S_2(m)$ (the entropy of the equal-weight superposition), $\langle S_2(1) \rangle$ (the mean single-state entropy) and $\Delta S_2(m)$ for each state class for the Rényi-2 entropy. Again, the shaded regions around each mean curve denote the interval $(-\sigma, +\sigma)$ of one standard deviation around the mean.

4.2.1 Haar-Random States:

In Figure 7, we plot all three quantities against the number of states m , and compare their behaviour for system sizes $N = 4, 6, 8, 10$.

We observe that $S_2(m)$ and $\langle S_2(1) \rangle$ both remain approximately constant for all values of m . Their exact values increase with system size N , which is consistent with our Page curve results from Section 4.1.1. Their difference $\Delta S_2(m)$ is centered at zero for all m and N , with only very minor fluctuations. This is an intuitive result; as Haar-random states are constructed by sampling the complex Gaussian distribution, any superposition of Haar-random states is itself approximately Haar-random, so the superposition entropy should match the mean entropy of its individual components.

For both $S_2(m)$ and $\Delta S_2(m)$, the number of states has no significant effect on the standard deviation. Meanwhile, $\langle S_2(1) \rangle$ shows a small decrease in its standard deviation for the first few values of m before stabilizing, an expected consequence of having a smaller sample size. Similarly as for the mean of ΔS_2 , the standard deviation decreases noticeably as we increase system size. Again, this corroborates superpositions of Haar-random states being Haar-random themselves.

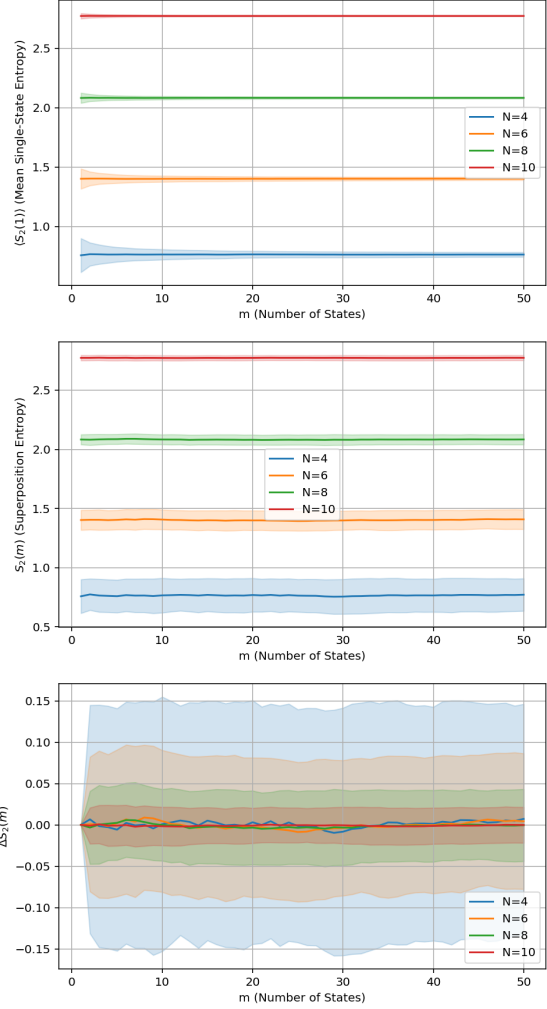
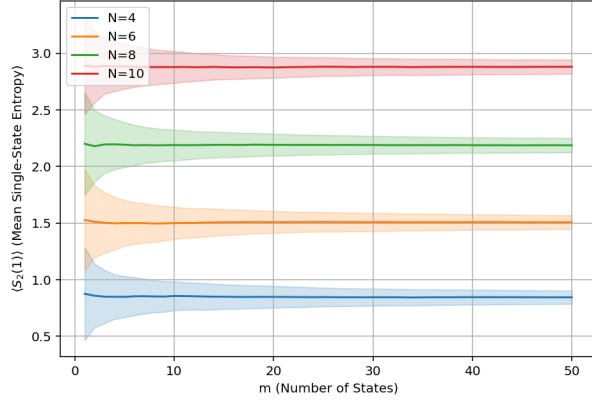


Figure 7: $S_2(m)$, $\langle S_2(1) \rangle$ and $\Delta S_2(m)$ for Haar-random states, measured for $N = 4, 6, 8, 10$ qubits, with 500 trials and up to 50 states.

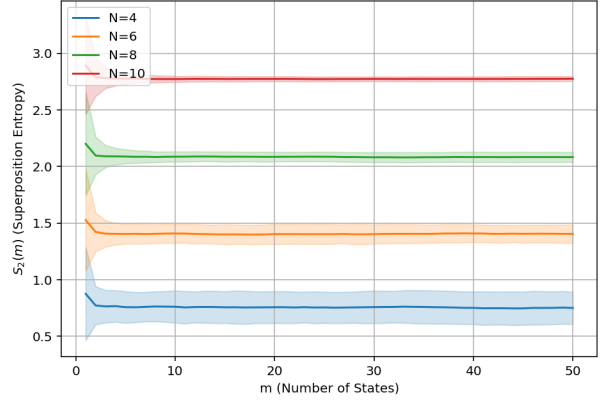
4.2.2 Clifford Stabilizer States:

In Figure 8 (a)-(c), we plot $S_2(m)$, $\langle S_2(1) \rangle$ and $\Delta S_2(m)$ against the number of states m for Clifford stabilizer states, for system sizes $N = 4, 6, 8, 10$. In Figure 8(d), we compare the mean single-state entropy for Haar-random states to the Clifford superposition entropy for $N = 8$.

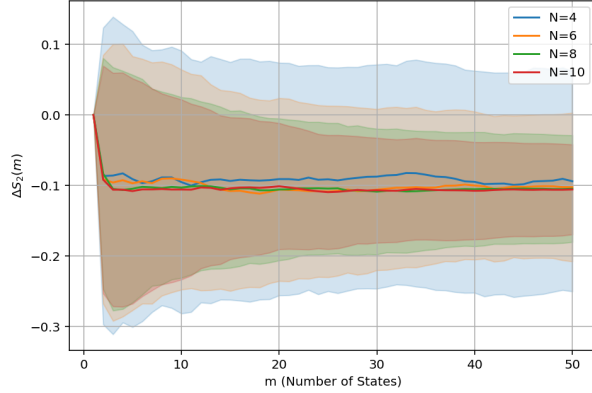
Similarly to Haar-random states, we observe that both $S_2(m)$ and $\langle S_2(1) \rangle$ remain relatively



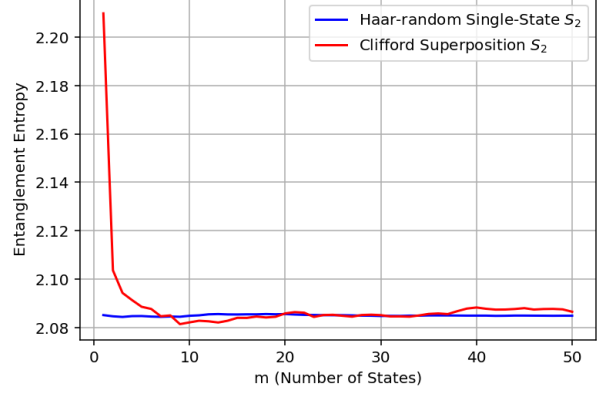
(a) $\langle S_2(1) \rangle$ for Clifford stabilizer states, measured for $N = 4, 6, 8, 10$ qubits, with 500 trials and up to 50 states.



(b) $S_2(m)$ for Clifford stabilizer states, measured for $N = 4, 6, 8, 10$ qubits, with 500 trials and up to 50 states.



(c) $\Delta S_2(m)$ for Clifford stabilizer states, measured for $N = 4, 6, 8, 10$ qubits, with 500 trials and up to 50 states.



(d) Direct comparison of $\langle S_2(1) \rangle$ Haar-random states, and $S_2(m)$ for Clifford stabilizer states, for $N = 8$, 500 trials and up to 50 states.

Figure 8: $\langle S_2(1) \rangle$, $S_2(m)$ and $\Delta S_2(m)$ for Clifford stabilizer states, with a direct comparison to the Haar-random single-state entropy in (d).

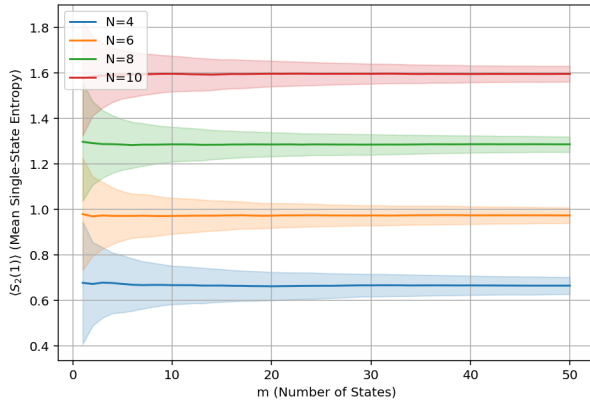
constant. Like before, we see a substantial increase in these quantities related to system size. However, unlike Haar-random states, $S_2(m)$ and $\langle S_2(1) \rangle$ carry a slight difference between them, with $\Delta S_2(m)$ quickly decreasing to ~ -0.1 and stabilizing there. This suggests that the equal-weight superposition of Clifford stabilizer states is inherently less entangled on average than its mean individual component.

This is likely a consequence of superposed Clifford states potentially lying outside the set of all Clifford stabilizer states. As we increase the number of states in our superposition, the resulting state will break the symmetries of Clifford stabilizer states, and more and more resemble a 'uniformly random' Haar-random state. This is illustrated explicitly in Figure 8(d), where a direct comparison shows that the Clifford superposition entropy quickly converges to the Haar-random single state entropy.

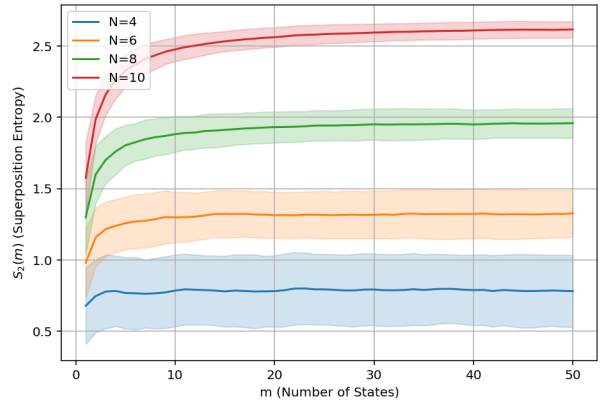
The standard deviations further support this idea. Notably, we observe a decrease in the standard deviation of $\Delta S_2(m)$ as we increase the number of states for all values of N . This is especially obvious for higher system sizes, with the standard deviation interval contracting far tighter for $N = 10$ than for $N = 4$. This is again a result of superpositions of Clifford stabilizer states becoming approximately Haar-random.

4.2.3 Number-Conserving States:

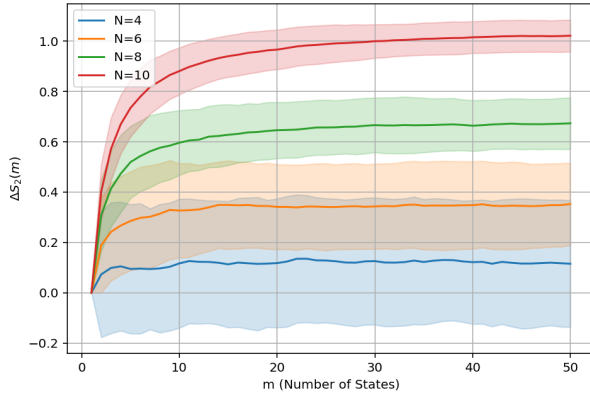
In Figure 9(a)-(c), we plot $S_2(m)$, $\langle S_2(1) \rangle$ and $\Delta S_2(m)$ against the number of states m for number-conserving states, for system sizes $N = 4, 6, 8, 10$. In Figure 9(d), we plot ΔS_2 for 500 trials with a logarithmic x-scale (i.e. taking $\ln(m)$ instead of m on the x-axis).



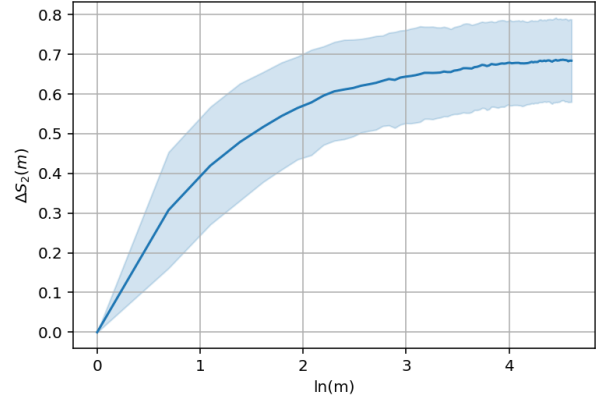
(a) $\langle S_2(1) \rangle$ for number-conserving states, measured for $N = 4, 6, 8, 10$ qubits, with 500 trials and up to 50 states.



(b) $S_2(m)$ for number-conserving states, measured for $N = 4, 6, 8, 10$ qubits, with 500 trials and up to 50 states.



(c) $\Delta S_2(m)$ for number-conserving states, measured for $N = 4, 6, 8, 10$ qubits, with 500 trials and up to 50 states.



(d) $\Delta S_2(m)$ for number-conserving states plotted against $\ln(m)$ for $N = 8$, with 500 trials and up to 100 states.

Figure 9: $\langle S_2(1) \rangle$, $S_2(m)$ and $\Delta S_2(m)$ for number-conserving states, with $\Delta S_2(m)$ plotted with a logarithmic x-scale in (d).

Number-conserving states show the clearest distinction from all 3 state families for both their mean and standard deviation. For all system sizes, $\langle S_2(1) \rangle$ remains constant with m , as expected. Although these values grow with N , they remain well below the mean single-state entropies of both Haar-random and Clifford states, as well as the maximum entropy S_{max} , agreeing with both the theory from Section 2.2.3 and our Page curve results from Section 4.1.3.

Unlike both Haar-random and Clifford stabilizer states, the superposition entropy $S_2(m)$ increases with m initially before saturating at a roughly constant value. The amount of states required for the curve plateaus depends on system size, as for instance, it takes a superposition of ~ 5 states to saturate for $N = 4$, but upwards of 50 states for $N = 10$ (as seen in Figure 9(b)). This could be a result of number-conserving superpositions being restricted to the same n -fermion sector, where once the sector is sufficiently sampled, the entropy cannot increase further.

In Figure 9(c), we observe a similar phenomenon for ΔS_2 . It initially appears that the growth of ΔS_2 as we increase m resembles a logarithmic growth with general form $f(x) = a + b \ln(x)$ for some $a, b \in \mathbb{R}$. However, in Figure 9(d), we observe that ΔS_2 does not increase proportionally with $\ln(m)$, ruling out logarithmic growth. We also note that the saturation value of ΔS_2 depends on system size, with it saturating at ~ 0.15 for $N = 4$, and ~ 0.35 for $N = 6$. Determining the overall trending behaviour and functional form of ΔS_2 requires further research, but is beyond the scope of this project.

The standard deviations of both $S_2(m)$ and ΔS_2 remain relatively constant across all values of m , but decrease as system size is increased. These standard deviations appear to be substantially larger than for Haar-random and Clifford stabilizer states, as we will confirm in Section 4.2.4.

4.2.4 State Family Comparison:

In Figure 10, we compare all $\langle S_2(1) \rangle$, $S_2(m)$ and $\Delta S_2(m)$ for all 3 state-classes.

In the $\langle S_2(1) \rangle$ plot, we observe that the mean single-state entropies of all state families remain approximately constant. The Clifford and Haar-random entropies are close, with the Clifford entropy slightly higher (in agreement with Figure 6 in Section 4.1.4). The number-conserving mean single-state entropy lags significantly behind these two, a direct consequence of the sub-maximal entropy of number-conserving states. The standard deviations are very different for each family; for Haar-random states we see a very tight spread, number-conserving states have a moderate spread initially and a slight decrease, while Clifford states have the largest initial variation and decrease substantially (although still have the largest deviation for higher values of m).

In the $S_2(m)$ plot, we see the Haar-random and Clifford superposition entropies match almost exactly for every value of m . Since $S_2(m)$ and $S_2(1)$ are approximately equal for Haar-

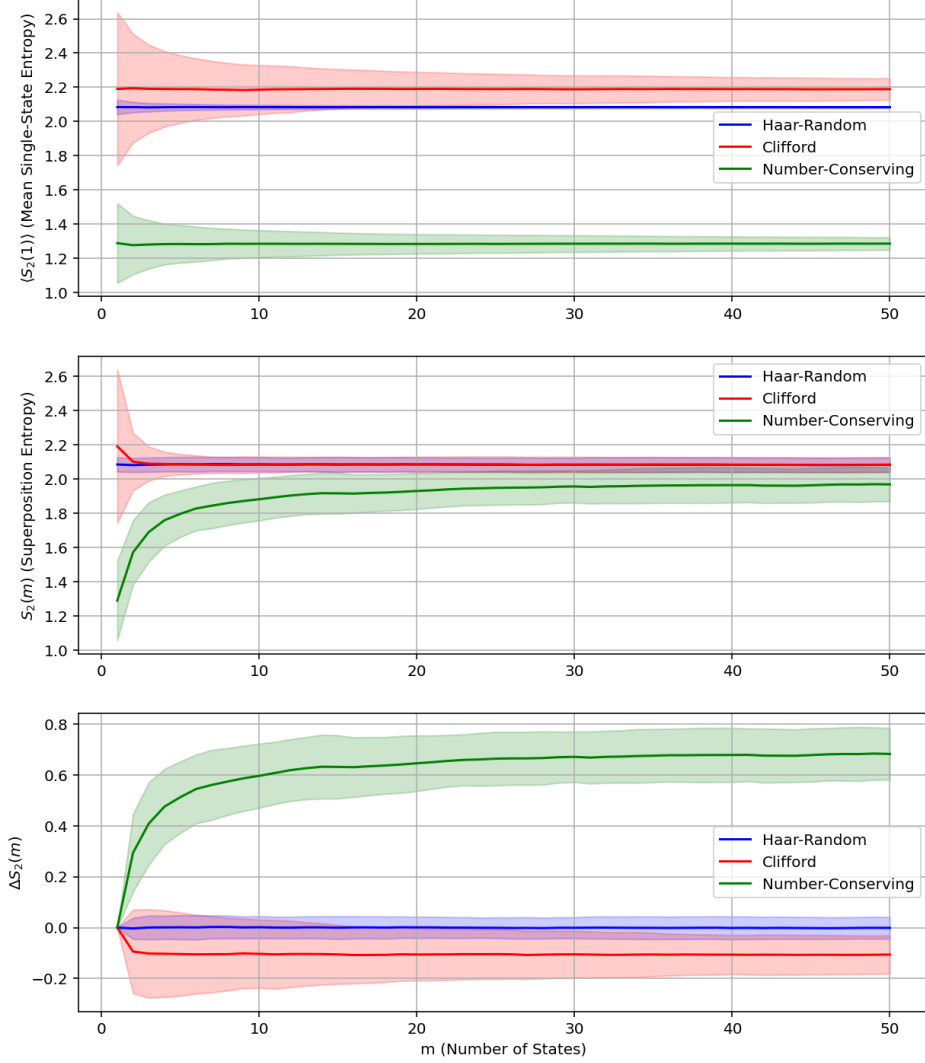


Figure 10: Comparisons of $\langle S_2(1) \rangle$, $S_2(m)$ and $\Delta S_2(m)$, for $N = 8$, 500 trials and up to 50 states.

random states, and since the Clifford superposition entropy converges to the Haar-random mean single-state entropy, it must also converge to the Haar-random superposition entropy. The number-conserving entropy starts significantly below both other state families and increases steadily, approaching but not reaching the other two states by $m = 50$. The standard deviation of number-conserving states here is noticeably larger than for either other state type (excluding small values of m , where once again the Cliffords have the highest standard deviation).

In the $\Delta S_2(m)$ plot, we observe that $\Delta S_2 \approx 0$ for Haar-random states, while $\Delta S_2 \approx -0.1$ for Clifford stabilizer states (excluding the initial drop from $\Delta S_2(m) = 0$ for the first few values of m). For number-conserving states, ΔS_2 rises to ~ 0.65 before saturating. The standard deviations are largest for number-conserving states, small for Haar-random states, and start off large for Cliffords, but decrease steadily with m .

5 Conclusion and Further Investigation:

In this project, we investigated the entanglement behaviour of superpositions of Haar-random, Clifford stabilizer states and number-conserving states through Page curve and superposition analyses.

In the Page curve analysis, we found that Haar-random and Clifford stabilizer states demonstrated near-maximal entanglement. The von Neumann and Rényi-2 entropies of both approach but crucially do not reach maximal entanglement $S_{max} = N_A \ln(2)$ as we increase system size, due to the Page difference explained in Section 2.1. Notably, Clifford stabilizer states have their von Neumann and Rényi-2 entropies exactly equal for all states, while the Rényi-2 and von Neumann entropies maintain an approximately constant difference for Haar-random states. Number-conserving states showed sub-maximal entanglement, with a near-constant ratio $\langle S_2 \rangle / \langle S \rangle \approx 0.81$ between both entropies.

In the superposition analysis, $\Delta S_2 \approx 0$ for Haar-random states, illustrating that taking superpositions of Haar-random states does not significantly affect their overall entanglement structure. For Clifford stabilizer states, ΔS_2 dropped to ≈ -0.1 and stabilized for all system sizes, with the Clifford superposition entropy approaching the Haar mean single-state entropy. Thus, equal-weight superpositions of Clifford stabilizer states show a small reduction in entanglement entropy compared to the mean single-state entropy of each component. Meanwhile, number-conserving states showed an increase in entropy, with ΔS_2 initially rising before saturating. Additionally, the saturation value of ΔS_2 for number-conserving states grows with increasing system size.

Through this project, we have constructed a toolkit which could be used for further investigation into the nature of entanglement of superposed states. Potential avenues for further investigation include studying the effect of subsystem size on the entanglement of superpositions, as well as investigating the behaviour of other state families (such as non-number-conserving states). Furthermore, a more in-depth analysis of the functional form of ΔS_2 for number-conserving states would likely prove intriguing, as our current results indicate it is not logarithmic, yet resembles a logarithmic form.

6 References:

- [1]: A. Einstein, B. Podolsky and N. Rosen, *Can Quantum-Mechanical Description of Physical Reality Be Considered Complete?*, Phys. Rev. **47**, 777 (1935).
- [2]: E. Schrödinger, *Discussion of probability relations between separated systems*, Math. Proc. Camb. Phil. Soc. 31, **555** (1935).
- [3]: D. Bohm, *A suggested interpretation of the quantum theory in terms of hidden variables*, Parts I and II, Phys. Rev. **85**, 166-179 (1952).

- [4]: J. S. Bell, *On the Einstein-Podolsky-Rosen paradox*, Phys. Phys. Fiz. **1**, 195 (1964).
- [5]: A. Aspect, P. Grangier and G. Roger, *Experimental Tests of Realistic Local Theories via Bell's Theorems*, Phys. Rev. Lett. **47**, 460-463 (1981).
- [6]: The Nobel Prize Organisation, *Nobel Prize in Physics 2022 - Summary*, <https://www.nobelprize.org/prizes/physics/2022/summary/> (accessed April 2026).
- [7]: J. Eisert, *Quantum Information Theory Lecture Notes, Chapter 5*, Freie Universität Berlin, <https://www.physik.fu-berlin.de/en/einrichtungen/ag/ag-eisert/teaching/ws20-21/QuantumInformationTheoryChapter5.pdf> (accessed April 2026).
- [8]: T. Debris-Alazard, *Introduction to Quantum Computer Science and Applications, Lecture 3*, École Polytechnique, <https://tdalazard.io/quantum.html> (accessed April 2026).
- [9]: J. Preskill, *Lecture Notes on Quantum Information*, Caltech, https://www.preskill.caltech.edu/ph219/chap2_15.pdf (2015).
- [10]: M. Dairyko et al, *Note on von Neumann and Rényi entropies of a Graph*, arXiv:1609.00420 (2016).
- [11]: D. N. Page, *Average entropy of a subsystem*, arXiv:gr-qc/9305007v2 (1993).
- [12]: D. Gottesman, *The Heisenberg Representation of Quantum Computers*, arXiv:quant-ph/9807006 (1998).
- [13]: Qiskit Documentation, IBM Qiskit, https://qiskit.qotlabs.org/docs/api/qiskit/quantum_info (accessed April 2026).
- [14]: S. Aaronson and D. Gottesman, *Improved Simulation of Stabilizer Circuits*, arXiv:quant-ph/0406196 (2008).
- [15]: F. Mezzadri, *How to Generate Random Matrices from the Classical Compact Groups*, arXiv:math-ph/0609050 (2006).

7 Code Appendix:

7.1 File 1 (RDM_Entropy_Calcs_and_Validation):

```
"""
This file contains the code for the reduced density matrix
calculator, both entropy calculators, and the
code for each validation test. This code corresponds to Sections
3.1, 3.2 and 3.4 of the report.
"""

import numpy as np
from State_Gens import Haar_random_generator,
    Clifford_stab_generator_v3, number_conserve_state_generator

bell_test=False #If True, tests the entropies of the 4 Bell states
prod_test= False #If True, tests the entropies of all product
    states of assigned sizes of qubits.
bonus_prod_test = False #If True, tests the entropies of product
    states  $(|00\rangle+|01\rangle)/\sqrt{2}$  &
     $\#(|00\rangle+|01\rangle+|10\rangle+|11\rangle)/2$ , two product states not in the Fock basis
    states.
subsys_test= False #If True, tests the entropies of a given state
    to show  $S(\rho_A)=S(\rho_B)$ 

#=====
#                               RDM AND ENTANGLEMENT ENTROPY CALCULATORS
#=====

def reduced_density_matrix(vec, qubits_kept, N, normal=False,
    print_ans=False):
    """
    --- Edited version of qiskit.quantum_info.partial_trace (removed
        the Qiskit Statevector wrapping,
    --- and removed the reverse index counting they use in that
        function)
    --- Calculates the partial trace for a vector WITHOUT building the
        density matrix using np.tensordot
    --- (using tensor reshaping)
    --- vec: NumPy array, state vector to find rdm of
    --- qubits_kept: List, integer values of qubits kept (0=1st
        qubit, 1=2nd qubit, etc.)
    --- N: Integer, total number of qubits
    --- normal: Boolean, normalizes the vector before tracing if True
    --- print_ans: Boolean, prints the density matrix if True
    """
```

```

"""
    if normal:
        vec /= np.linalg.norm(vec) #Optional normalization check
    N_A = len(qubits_kept)
    qubits_traced = [i for i in range(N) if i not in qubits_kept]
        #List of qubits traced out
    arr = vec.reshape([2]*N)
    rho = np.tensordot(arr, arr.conj(), axes=(qubits_traced,
        qubits_traced))
    #This calculates the outer product of the rows of arr with
        indices in kept qubits
    # (arr*arr.conj()), and sums the rows of arr with indices in
        qubits_traced
    rho_A = rho.reshape(2**N_A, 2**N_A) #Reshape back into density
        matrix form
    if print_ans:
        print(rho_A)
    return rho_A

def von-Neumann-entropy(rho, print_ans=True):
    """
    --- Returns the von-Neumann entropy for a given reduced density
        matrix by taking its eigenvalues and building  $S = -\sum(\lambda_i \ln \lambda_i)$ ,  $\lambda_i$  being the eigenvalues of rho.
    --- rho: NumPy array, the reduced density matrix
    --- print_ans: Boolean, prints answers if True
    """
    eigvals = np.linalg.eigvalsh(rho) #eigvalsh used as every
        density matrix is Hermitean
    eigvals = eigvals[np.abs(eigvals)>1e-12] #Safety check,
        eliminates eigenvalues at 0 (causes errors with log(eig))
    S = -sum(eigvals * np.log(eigvals)) #von Neumann entropy
    if print_ans:
        print("Von-Neumann-Entropy:", S)
    return S

def Rényi-2-entropy(rho, print_ans=True):
    """
    --- Returns the Rényi-2 entropy for a given reduced density matrix
        , by calculating  $S = -\ln(\text{Tr}(\rho_A^2))$ 
    --- rho: NumPy array, reduced density matrix
    --- print_ans: Boolean, prints answers if True
    """
    S = -np.real(np.log(np.trace(rho @ rho))) #Rényi-2 entropy
    if print_ans:

```

```

        print("Rényi-2-Entropy:" , S)
    return S

```

```

#=====
#                               FUNCTIONS FOR VALIDATION TESTS
#=====

def generate_basis_states_per_qubit_no(N, print_ans=True):
    """
    -----Generates-and-returns-all-computational-basis-states-per-qubit
             -number-as-a-list.
    -----N:-Integer,-dimension
    -----print_ans:-Boolean,-prints-answers-if-True
    -----"""
    state_list = []
    for index in range(0, 2**N): #2**N used as range excludes last
        value
        psi=np.zeros(2**N) #|0>^N
        psi[index]=1 #Generates every matrix with a 1 in a single
            entry, and 0 in all others (i.e.)
        state_list.append(psi)
    if print_ans:
        print(state_list)
    return state_list

def single_product_test(state, N, qubits_kept, print_all=False,
    TOL=1e-15):
    """
    -----Tests-the-von-Neumann/Rényi-2-entropies-for-a-given-product-
             state,-for-a-given-subsystem.
    -----Returns-False-if-either-entropy-is-non-zero,-within-a-given-
             tolerance.
    -----state:-NumPy-array,-the-state-to-be-tested.
    -----N:-Integer,-number-of-qubits.
    -----qubits_kept:-List,-integer-positions-of-qubits-in-subsystem-(
             index-0-for-the-leftmost-one)
    -----print_all:-Boolean,-prints-all-entropies-calculated-if-True
    -----TOL:-Float,-cutoff-point-for-floating-point-errors-(i.e.-if-|a
             |<TOL,-we-treat-a=0)
    -----"""
    if print_all:
        print(state)
    rho = reduced_density_matrix(vec=state, qubits_kept=
        qubits_kept, N=N, normal=False,
        print_ans=False) #Reduced density matrix

```

```

v=von_Neumann_entropy(rho, print_ans=print_all)
r=Rényi_2_entropy(rho, print_ans=print_all)
if (abs(v)>TOL) or (abs(r)>TOL): #Only counts an entropy value
    as non-zero if its absolute value is greater than the
    tolerance.
    return False #Will give an error in main test if False
else:
    return True #Ok if True

def subsystem_test(state, N, A_qubits, TOL=1e-12, print_all=False)
:
    """
    ---- Calculates S(rho_A), S(rho_B) for the von Neumann and Rényi-2
    entropies
    ---- for a given pure state, and subsystems A and B.
    ---- Returns False if the two are not equal within a certain
    tolerance, and returns True if they match.
    ---- state: Numpy Array, state being tested.
    ---- N: Integer, number of qubits.
    ---- A_qubits: List, integer positions of qubits for subsystem A (
    starts at 0)
    ---- TOL: Float, tolerance for error
    ---- print_all: Boolean, prints everything if True
    ---- Note the qubits of B are assumed to be all qubits not in A.
    ---- """
    B_qubits = [num for num in range(N) if num not in A_qubits] #
        Qubits in subsystem B
    #Calculating S(rho_A)
    rho_A = reduced_density_matrix(vec=state, qubits_kept=A_qubits
        , N=N, normal=False, print_ans=False) #RDM of A
    vn1 = von_Neumann_entropy(rho_A, print_ans=False)
    r1 = Rényi_2_entropy(rho_A, print_ans=False)
    #Calculating S(rho_B)
    rho_B = reduced_density_matrix(vec=state, qubits_kept=B_qubits
        , N=N, normal=False, print_ans=False) #RDM of B
    vn2 = von_Neumann_entropy(rho_B, print_ans=False)
    r2 = Rényi_2_entropy(rho_B, print_ans=False)
    if print_all:
        print(state)
        print("Entropies for Subsystem A:")
        print("Von Neumann Entropy for Subsystem A: ", vn1, "Rényi
            -2 Entropy for Subsystem A: ", r1)
        print("Entropies for Subsystem B:")
        print("Von Neumann Entropy for Subsystem B: ", vn2, "Rényi
            -2 Entropy for Subsystem B: ", r2)

```



```

        print("\n")
#Safety check
if (abs(vn2-vn1)>TOL) or (abs(r2-r1)>TOL):
    return False #If False, the entropies don't match
else:
    return True #If True, the entropies are ok

=====
#
#                               VALIDATION TESTS
#
=====

#Bell State Tests: Tests the von Neumann/Rényi-2 entropies for the
4 Bell states.
bell_states = [np.array([1/np.sqrt(2), 0, 0, 1/np.sqrt(2)]),
               np.array([1/np.sqrt(2), 0, 0, -1/np.sqrt(2)]),
               np.array([0, 1/np.sqrt(2), 1/np.sqrt(2), 0]),
               np.array([0, 1/np.sqrt(2), -1/np.sqrt(2), 0])]
if bell_test:
    for state in bell_states:
        print(state)
        print("Tests-on-First-Qubit-Subsystem:")
        rho_A = reduced_density_matrix(vec=state, qubits_kept=[0],
                                         N=2, normal=False, print_ans=False)
        von_Neumann_entropy(rho_A, print_ans=True)
        Rényi_2_entropy(rho_A, print_ans=True)
        print("Tests-on-Second-Qubit-Subsystem:")
        rho_B = reduced_density_matrix(vec=state, qubits_kept=[1],
                                         N=2, normal=False, print_ans=False)
        von_Neumann_entropy(rho_B, print_ans=True)
        Rényi_2_entropy(rho_B, print_ans=True)
        print("\n")
#Both von Neumann and Rényi-2 entropies give  $S=\ln(2)$  for each
Bell state.

#Product Test: This tests all computational basis states for a
specified number of qubits, and a specified subsystem size, and
shows that
#they all have 0 von Neumann and Rényi-2 entropy.
if prod_test:
    tested_no_qubits=[1, 2, 3, 4, 5, 6] #This list can be changed
    to test different system sizes.
    subsystem_A_qubits=[0] #This list can be changed to test
    different subsystem qubits.
    print_all=False #This can be changed, prints all entropies and
    states if True.

```

```

all_passed=True #Used to check if every product state has 0
entropies
for N in tested_no_qubits:
    N_qubit_states = generate_basis_states_per_qubit_no(N=N,
        print_ans=False)
    print(f"{N}-qubit-tests:") if print_all else None #Just
        for print_all=True
    for state in N_qubit_states:
        #Individual tests for each product state for a fixed N
        result = single_product_test(state, N, qubits_kept=
            subsystem_A_qubits, print_all=print_all)
        if result==False:
            all_passed=False
            break #Safety check, breaks everything if one of
                the entropies is non-zero
        print("\n") if print_all else None #Just for print_all
            =True
    if all_passed==False:
        print("Product-Test-Failed:-Not-all-States-have-0-
            Entropies")
    else:
        print("Product-Test-Success:-0-Entropies-for-Every-State-
            Tested")
#Tested this with every system size of <=6 qubits, and every
possible combination of qubits preserved
#for the subsystem, it still gives both entropies to be 0 for
every state.

#Bonus Product Test: Tests the entropies of product states
(|00>+|01>)/sqrt(2) & (|00>+|01>+|10>+|11>)/2
if bonus_prod_test:
    state1 = np.array([1/np.sqrt(2), 1/np.sqrt(2), 0, 0]) #
        (|00>+|01>)/sqrt(2)
    state2 = np.array([1/2, 1/2, 1/2, 1/2]) #(|00>+|01>+|10>+|11>)
        /2
    #Tests on first state for both single-qubit subsystems
    test1a = single_product_test(state1, N=2, qubits_kept=[0],
        print_all=False)
    test1b = single_product_test(state1, N=2, qubits_kept=[1],
        print_all=False)
    #Tests for second state on both single-qubit subsystems
    test2a = single_product_test(state2, N=2, qubits_kept=[0],
        print_all=False)

```

```

test2b = single_product_test(state2, N=2, qubits_kept=[1],
    print_all=False)
if not (test1a==test1b==test2a==test2b==True):
    print("Error:-Not-All-Entropies-Equal-to-0") #Error
    message if entropies not 0
else:
    print("Both-States-have-Entropy-0-With-Both-Measures") #
    All ok

#Subsystem Test:
if subsys_test:
    #Samples to be tested from the Haar-random, Clifford and
    number-conserving families.
    #Specific details here can be changed, e.g. number of qubits,
    or amount of states taken
    print_all = False #If True, will print each state and its
    entropies of both subsystems for both measures.
    Haar_test = Haar_random_generator(no_states=100, N=4) #Haar-
    random states
    Cliff_test = Clifford_stab_generator_v3(no_states=100, N=4) #
    Clifford states
    conserve_test = number_conserve_state_generator(no_states=100,
        N=4, n=2) #Number-conserving states
    A_qubits = [0,2] #Qubits in subsystem A, can be changed
    N=4 #Number of qubits in the system, can be changed.
    total_test = Haar_test + Cliff_test + conserve_test #Combining
    all 3 types of state into one test
    for state in total_test:
        result = subsystem_test(np.array(state), N, A_qubits,
            print_all=print_all)
        if result==False: #If any mismatch for any tested state,
            breaks the whole thing.
            print("Error:-Mismatch-in-Entropies-Between-Subsystems
                ")
            break
    else:
        print("Subsystem-Test-Success:-All-Tested-Entropies-Match"
            )
#From bell_test and prod_test, both the von Neumann and Rényi-2
    entropies are identical for product
#states and Bell states. Haar-random, Clifford and number-
    conserving states are
#accurate up to  $10^{-12}$  for 4 qubits, and only show occasional
    errors for  $10^{-15}$  tolerance from testing.

```

7.2 File 2 (State_Gens):

```
"""
This file contains the code of all state generators, (i.e. for
    Haar-random, Clifford stabilizer and number-conserving states.)
This corresponds to Section 3.3 of the report.
"""

import numpy as np
import stim
import scipy as sp
import qiskit

#Pauli spin matrices and spin operators (used for get_c_sparse)
X = np.array([[0, 1], [1, 0]])
Y = np.array([[0, -1j], [1j, 0]])
Z = np.array([[1, 0], [0, -1]])
I_sparse = sp.sparse.identity(2, format='csr')
Z_sparse = sp.sparse.csr_matrix(Z)
sigma_plus = 0.5 * sp.sparse.csr_matrix(X - 1j*Y) #sigma_plus
    |0>=|1>
sigma_minus = 0.5 * sp.sparse.csr_matrix(X + 1j*Y) #sigma_minus
    |1>=|0>

#=====
#                               HAAR-RANDOM STATE GENERATOR
#=====

def Haar_random_generator(no_states, N, seed=None, print_ans=False
    ):
    """
    ----Generates a collection of Haar-random states by sampling real/
        complex parts of each value
    ----from the Gaussian distribution.
    ----no_states: Integer, number of states generated.
    ----N: Integer, number of qubits.
    ----seed: Integer/None, determines the normal distribution to
        sample real/complex values from. (None by default)
    ----print_ans: Boolean, prints answers if True (False by default)
    ----Returns the results as NumPy Arrays in a list
    ----"""
    rng = np.random.default_rng(seed) #Random number generation
    results=[] #Placeholder list for results, every state added to
        this
    for _ in range(no_states):
```

```

    real_vals = rng.normal(loc=0, scale=1, size=2**N) #Random
        real parts
    complex_vals = rng.normal(loc=0, scale=1, size=2**N) #
        Random complex parts
    psi = real_vals+(complex_vals*1j)
    psi /= np.linalg.norm(psi) #Normalizing
    results.append(psi)
    if print_ans: #Print all states if True
        print(results)
    return results

```

```

#
#                CLIFFORD STABILIZER STATE GENERATORS
#

```

```

def Clifford_stab_generator_v1(no_states, N, gate_range, seed=None
    , print_ans=False):
    """
    ---- Generates a random sample of Clifford stabilizer states. This
        version picks a random number from a range of numbers
    ---- (representing how many gates to apply), and applies that
        amount of random H,S and CNOT gates.
    ---- Added rng for number of gates applied, and pre-generated both
        the number of gates applied and the gates per state for speed.
    ---- NOTE: STIM ALWAYS PICKS THE SAME PHASE UNDER STIM.
        TABLEAUSIMULATOR AND STIM.TABLEAU.TO.STATE.VECTOR, SO THIS IS
        NOT ACCURATE
    ---- This version is NOT used in the final report.
    ---- no_states: Integer, number of states generated
    ---- N: Integer, number of qubits
    ---- gate_range: List, should contain TWO INTEGERS, for the min and
        max numbers respectively of gates to apply to ground state
    ---- Any other values in gate_range will be ignored.
    ---- seed: Integer, used for reproducibility, None by default
    ---- print_ans: Boolean, prints answers if True
    ---- """
    rng = np.random.default_rng(seed) #Random number generation
    results=[] #Empty list for results
    no_gates_list = rng.integers(gate_range[0], gate_range[1],
        size=no_states) #Pre-generate all amounts of gates to apply.
    for i in range(no_states):
        circuit=stim.Circuit() #Holds instructions for circuit,
            will be made of Hadamard, phase and CNOT gates, applied
            to ground state.
        no_gates = no_gates_list[i]

```

```

#Main Loop to Apply Random Gates
gate_type = rng.choice(["H", "S", "CNOT"], size=no_gates)
#Pre-generate all gates to apply for each state
for j in range(no_gates):
    if gate_type[j] in ["H", "S"]:
        rng_qubit = int(rng.integers(0, N)) #Random chosen
        target_qubit (int to remove 1-element NumPy
        array)
        circuit.append(gate_type[j], [rng_qubit]) #Add the
        random gate (rng_qubit as list for circuit.
        append)
    else: #Adding CNOT Gate
        rng_qubit = rng.integers(0, N, size=2) #2 qubits
        chosen (target and control)
        while rng_qubit[0]==rng_qubit[1]: #Safety check to
        prevent repeated indices for CNOT gate.
            rng_qubit[1]=rng.integers(0, N)
        circuit.append(gate_type[j], list(rng_qubit))
tableau = stim.TableauSimulator() #Makes tableau of the
circuit
tableau.do(circuit) #Applies everything to ground state
state = tableau.state_vector() #Resulting Clifford state
results.append(state)
if print_ans:
    print(results)
return results

def Clifford_stab_generator_v2(no_states, N, print_ans=False):
    """
    ----An alternate function to try to generate Clifford stabilizer
    states.- This uses stim's Tableau.Random function to ensure a
    uniform sample across the Clifford stabilizer states.
    ----NOTE: STIM ALWAYS PICKS THE SAME PHASE UNDER STIM.
    TABLEAUSIMULATOR AND STIM.TABLEAU.TO_STATE_VECTOR, SO THIS IS
    NOT ACCURATE
    ----This is NOT used in the final report.
    ----no_states: Integer, number of states generated
    ----N: Integer, number of qubits
    ----print_ans: Boolean, prints answers if True
    ----"""
    results = [] #List for states generated
    for _ in range(no_states):
        random_Clifford = stim.Tableau.random(N) #Random Clifford
        operation
        sim = stim.TableauSimulator() #|0>^N

```

```

        sim.do_tableau(random_Clifford, list(range(N))) #Apply the
            Clifford operation to the vacuum state.
        results.append(sim.state_vector())
    if print_ans:
        print(results)
    return results

def Clifford_stab_generator_v3(no_states, N, print_ans=False, seed
=None):
    """
    ----This is the final (and correct) generator for Clifford
    stabilizer states. This uses Qiskit's quantum_info.
    random_Clifford function, which considers all phases properly,
    and samples the Clifford stabilizer states uniformly at random.
    ----no_states: Integer, number of states to generate.
    ----N: Integer, number of qubits
    ----print_ans: Boolean, prints answer if True (False by default)
    ----seed: Integer, used for reproducibility (None by default)
    ----"""
    rng = np.random.default_rng(seed) #random_clifford actually
        uses this.
    results = [] #Empty list for states generated.
    psi0 = qiskit.quantum_info.Statevector.from_label('0' * N) #
        |0>^N, pre-generated for efficiency
    for _ in range(no_states):
        Clifford_operation = qiskit.quantum_info.random_clifford(N
            , seed=rng) #Random Clifford operation
        circuit = Clifford_operation.to_circuit() #Decomposes this
            into H,S,CNOT Circuit (more efficient)
        state = psi0.evolve(circuit) # Apply Clifford to |0>^N
        results.append(state.data) #Adds NumPy array to the list
    if print_ans:
        print(results)
    return results

#=====
# NUMBER-CONSERVING STATE GENERATOR (AND SUPPORTING FUNCTIONS)
#=====

def generate_random_unitary(d, seed=None, print_ans=False):
    """
    ----Mezzadri's method of generating random unitary matrices.
    Samples independent, identically distributed numbers from the
    complex Gaussian distribution, and applies QR decomposition.
    ----Returns a random unitary matrix.

```

```

----d: Integer, size of matrix (d, d)
----seed: Integer, seed for real/complex gaussian distribution (
    None by default)
----print_ans: Boolean, prints unitary generated if True (False by
    default)
----"""

rng= np.random.default_rng(seed) #Random number generation
real_vals = rng.normal(loc=0, scale=1, size=(d, d)) #Random
    real parts
complex_vals = rng.normal(loc=0, scale=1, size=(d, d)) #Random
    complex parts
matrix = real_vals + (1j*complex_vals)
q, r = sp.linalg.qr(matrix) #QR decomposition
lam = np.diagonal(r) #Diagonalizing the matrix
ph = lam/np.abs(lam) #Dividing each value by its absolute
    value
q = np.multiply(q, ph, q) #This matrix is random w.r.t. Haar-
    measure
if print_ans:
    print(q)
return q

def get_c_sparse(N, i, dagger, print_ans=False):
    """
    ---- Builds and returns a Jordan-Wigner operator as a tensor
        product of Z operators and sigma +/- .
    ---- Done through sparse matrices for efficiency.
    ---- N: Integer, number of qubits
    ---- i: Integer, index of Jordan-Wigner operator (This counts from
        1 to N, e.g. 1=1st qubit, etc.)
    ---- dagger: Boolean, gives c^+ if True, and c if False
    ---- print_ans: Boolean, prints the operator if True (False by
        default)
    ---- """

    operators = [] #List to contain all operators to multiply
        together to get c/c^+
    for k in range(1, N+1): #Subscripts 1 to N
        if k<i:
            operators.append(Z_sparse)
        elif k==i:
            if dagger:
                operators.append(sigma_plus) #Adds fermion
            else:
                operators.append(sigma_minus) #Removes fermion
        else: #k>i

```



```

        operators.append(I_sparse)
result = operators[0] #Initial matrix to multiply all other
        matrices onto
for vector in operators[1::]:
    result=sp.sparse.kron(result , vector , format='csr') #
        Individual tensor product
if print_ans:
    print(result)
return result

def number_conserve_state_generator(no_states , N, n, seed=None,
print_ans=False):
    """
    ----Generates and returns a list of number-conserving free-fermion
    -states.
    ----no_states: Integer , number of states generated
    ----N: Integer , number of qubits
    ----n: Integer , number of particles (0<n<N)
    ----seed: Integer , used for reproducibility (None by default)
    ----print_ans: Boolean , prints answers if True (False by default)
    ----"""

    state_list=[] #Empty list for states generated
    c_dag_ops=[get_c_sparse(N, k, dagger=True, print_ans=False)
        for k in range(1, N+1)] #Pre-built to save time
    vacuum_state = np.zeros(2**N, dtype=complex)
    vacuum_state[0] = 1 #|0>^N, Pre-generated for efficiency
    for i in range(no_states): #Iterates for a chosen no of states
        state = vacuum_state
        if seed!=None:
            seed += 1 #This ensures different unitaries for each
                beta operator
        V = generate_random_unitary(N, seed=seed , print_ans=False)
            #(N, N) matrix
        for site in range(0, n): #Site taken as 0-N, NOT 1-N+1
            beta = sum(V[j, site] * c_dag_ops[j] for j in range(N)
                ) #One beta operator
            state = beta @ state
        state /= np.linalg.norm(state) #Normalization of state
            afterwards
        state_list.append(state)
    if print_ans:
        print(state_list)
    return state_list

```

7.3 File 3 (Page_Curves):

```
"""
This function contains all functions to plot the Page curves for
each state family. This corresponds to Sections 3.5 and 4.1 of
the report.
"""

import numpy as np
import matplotlib.pyplot as plt
from RDM_Entropy_Calcs_and_Validation import
    reduced_density_matrix, von_Neumann_entropy, Rényi_2_entropy
from State_Gens import Haar_random_generator,
    Clifford_stab_generator_v3, number_conserve_state_generator

Haar_curve_plots = False #If True, plots the von Neumann and Rényi
    -2 entropies of 1000 Haar-random states for N=6,8,12,16.
#Also prints the mean and standard deviation at N/2 for each N
    value.
conserve_curve_plots = False #If True, plots the von Neumann and R
    ényi-2 entropies of 1000 number-conserving states for N
    =6,8,12,16.
#Also prints the mean and standard deviation at N/2 for each N
    value.
Clifford_curve_plots = False #If True, plots the von Neumann and R
    ényi-2 entropies of 1000 number-conserving states for N
    =6,8,12,16.
#Also prints the mean and standard deviation at N/2 for each N
    value.
indiv_Clifford_plots = False #If True, plots the entropies of 6
    individual 12-qubit Clifford stabilizer states.
S_max_comparison = False #If True, plots the von Neumann entropies
    of all 3 state families vs  $N_A \ln(2)$  for N=8,16
peak_comparison = False #If True, plots the peaks of each Page
    curve for N=4,6,8,10.

#=====
#                               PAGE CURVE VALUE CALCULATOR
#=====

def page_curve_vals(N, state_list, Rényi, von_Neumann, std):
    """
    --- This function gives the values to plot the Page Curve of the
        average entanglement entropy.
    --- Variables are included to decide what entropies to find the

```

```

    value of (Rényi-2 and/or von-Neumann) and to decide if standard
    deviations are included.
    """
    We always split the system into two subsystems of the N_A
    leftmost and N-N_A rightmost qubits, taking the smallest
    subsystem of the two.
    This returns (in order): List of Rényi-2 mean entropies, list
    of von-Neumann mean entropies,
    list of Rényi-2 standard deviation of the entropy (only if std
    ==True), list of von-Neumann
    standard deviation of the entropy (only if std==True)
    All 4 lists must always be assigned as variables, but will be
    empty unless their corresponding
    variable is True
    N: number of qubits (subsystem sizes are taken as 1,2,...,N-1)
    state_list: List, inputted states
    Rényi: Boolean, finds the values of the Rényi-2 entropy if
    True
    von-Neumann: Boolean, finds the values of the von-Neumann
    entropy if True
    std: Boolean, returns the standard deviation values of all
    entropies considered if True
    """
    N_A_vals = range(1, N) #1 to N-1 qubits in A
    S2_means=[] #Rényi-2 mean entropy list
    S2_stds = [] #Rényi-2 standard deviation list
    S_means = [] #von Neumann entropy list
    S_stds = [] #von Neumann standard deviation list
    for N_A in N_A_vals:
        #For optimisation, keep the qubits in the smaller
        subsystem (as S(rho-A)=S(rho-B))
        if N_A <= (N//2):
            qubits_kept = [i for i in range(0, N_A)] #First N_A
            qubits kept.
        else:
            qubits_kept = [i for i in range(N_A, N)]
        #All relevant density matrices
        rdm_list = [reduced_density_matrix(vec=state, qubits_kept=
            qubits_kept,
            N=N, normal=False, print_ans=False) for state
            in state_list]
        if Rényi:
            #All Rényi-2 entropies for each rdm (only calculated
            if Rényi==True)
            Rényi_2_list=[Rényi_2_entropy(rho, print_ans=False)
                for rho in rdm_list]

```

```

        S2_means.append(np.mean(Rényi_2_list)) #Take the mean
        entropy of all states
        S2_stds.append(np.std(Rényi_2_list)) if std else None
    if von_Neumann:
        #All von Neumann entropies for each rdm (only
        calculated if von_Neumann==True)
        von_Neumann_list=[von_Neumann_entropy(rho, print_ans=
            False) for rho in rdm_list]
        S_means.append(np.mean(von_Neumann_list))
        S_stds.append(np.std(von_Neumann_list)) if std else
            None
    return S2_means, S_means, S2_stds, S_stds

#=====
#                               PAGE CURVE PLOTTING FUNCTIONS
#=====

def plot_page_curve(N, state_list, Rényi=True, von_Neumann=True,
    errors=False, title=None,
                        print_peak_vals=True):
    """
    ---- Plots the Page curve of one or both entropies for a single
        list of states.
    ---- N: Integer, number of qubits
    ---- state_list: List, contains inputted states to test.
    ---- Rényi: Boolean, plots the Rényi-2 page curve if True
    ---- von_Neumann: Boolean, plots the von Neumann page curve if True
    ---- errors: Boolean, plots the error lines (mean +/- standard
        deviation) for each plotted entropy
    ---- title: String, title for the plot, does nothing if None (None
        by default)
    ---- print_peak_vals: Boolean, if True, prints the values at the
        peak of each Page curve.
    ---- """
    plt.figure(figsize=(6.4,4.2)) #Subfigures
    N_A_vals = range(1, N) #x-axis
    S2_vals, S_vals, S2_stds, S_stds = page_curve_vals(N,
        state_list, Rényi, von_Neumann, std=errors)
    if Rényi:
        plt.plot(N_A_vals, S2_vals, label="Rényi-2", color="blue")
        #Plotting average Rényi
    if errors:
        S2_error_up = np.add(S2_vals, S2_stds)
        S2_error_down = np.subtract(S2_vals, S2_stds)
        plt.fill_between(N_A_vals, S2_error_down, S2_error_up,

```

```

        color="blue", alpha=0.2)
#Print S2's mean and (OPTIONALLY) std dev at N_A=N//2
if print_peak_vals:
    print("S2-Mean:-", S2_vals[N//2-1])
    if errors:
        print("S2-Deviation:-", S2_stds[N//2-1])
if von_Neumann:
    plt.plot(N_A_vals, S_vals, label="von-Neumann", color="red")
    #Plotting average von Neumann
    if errors:
        S_error_up = np.add(S_vals, S_stds)
        S_error_down = np.subtract(S_vals, S_stds)
        plt.fill_between(N_A_vals, S_error_down, S_error_up,
            color="red", alpha=0.2)
#Print S2's mean and (OPTIONALLY) std dev at N_A=N//2
if print_peak_vals:
    print("S-Mean:-", S_vals[N//2-1])
    if errors:
        print("S-Deviation:-", S_stds[N//2-1])
plt.xlabel("$N_A$")
plt.ylabel("Entanglement-Entropy")
plt.grid()
plt.legend()
if title!=None: #Optional Title
    plt.title(title)
plt.show()

def page_curve_state_comparison_plot(N, list_of_families,
    family_names, Rényi=True,
                                von_Neumann=True, S_max=False,
                                title=None, label_entropy=False):
    """
    -----Plots the Page-Curves of multiple-separate-state-families-into
    -a-single-plot-for-a-single-system-size-N.
    -----This-function-also-has-the-option-to-plot  $N_A \ln(2)$ -to-compare
    -these-entropies-to-the-theoretical-maximum.
    -----N:-Integer, -number-of-qubits
    -----list_of_families:-List, -contains-multiple-LISTS, -each-list-is-
    -treated-as-a-single-state-family-to-test-and-plot
    -----family_names:-List, -contains-multiple-STRINGS, -each-string-
    -will-be-the-label-of-the-plot-made-by-its-corresponding
    -----family.
    -----Rényi:-Boolean, -plots-Rényi-2-entropy-for-ALL-families-if-True
    -----von-Neumann:-Boolean, -plots-von-Neumann-entropy-for-ALL-

```

```

    families-if-True
----S_max:-Boolean,-plots-N_A-ln(2)-if-True
----title:-String,-gives-title-to-the-plot-(None-by-default)
----label_entropy:-Boolean,-adds-specified-entropy-names-to-labels
    -on-the-plot-if-True-(False-by-default)
----"""

    plt.figure(figsize=(6.4,4.2)) #Subplots
    N_A_vals = range(1, N)
    for i, state_list in enumerate(list_of_families):
        S2_vals, S_vals, S2_stds, S_stds = page_curve_vals(N,
            state_list, Rényi, von-Neumann, std=False) #Mean/
            deviation values for both entropies
        if Rényi:
            plt.plot(N_A_vals, S2_vals, label=family_names[i]+(":
                Rényi-2" if label_entropy else "")) #Plotting
                average Rényi
        if von-Neumann:
            plt.plot(N_A_vals, S_vals, label=family_names[i]+(":
                von-Neumann" if label_entropy else "")) #Plotting
                average von Neumann
    #Plotting N_A ln(2) if S_max==True
    if S_max:
        S_max_vals = [min(N_A, N-N_A)*np.log(2) for N_A in
            N_A_vals] #N_A ln(2), always takes N_A as smaller
            subsystem
        plt.plot(N_A_vals, S_max_vals, label="$S_{max}=N_A$-ln(2)"
            , ls="—")
    plt.xlabel("$N_A$")
    plt.ylabel("Entanglement-Entropy")
    plt.grid()
    plt.legend(loc='lower-center')
    if title!=None:
        plt.title(title)
    plt.show()

def page_curve_indiv_state_entropy_comparison_subplots(N,
    state_list, title=None):
    """
    ----This-function-plots-the-Page-Curves-of-both-the-Rényi-2-and-
        von-Neumann-entropies-of-individual-states-splitting-the-results
        -into-two-plots-based-on-which-entropy-is-measured.
    ----This-function-is-only-used-for-the-Clifford-stabilizer-states,
        -as-they-overlap-exactly.
    ----N:-Integer,-number-of-qubits
    ----state_list:-List,-contains-arrays-of-all-states-to-measure-the

```

```

    - entropies of (small lists are
    - recommended, or your graph will be very cluttered)
    - title: String, gives the plot a title (None by default)
    - """
    N_A_vals = range(1,N)
    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(15,5)) #Subplots
    for i, state in enumerate(state_list): #Runs through each
        state
        S2_vals, S_vals, S2_stds, S_stds = page_curve_vals(N, [
            state], Rényi=True,
            von_Neumann=True, std=False) #Finds the 'mean'
            entropies of one state, i.e. just its entropy
        #Plotting the entropies of each state on separate subplots
        ax1.plot(N_A_vals, S2_vals, alpha=0.7, label=f"State-{i+1}
            ")
        ax2.plot(N_A_vals, S_vals, alpha=0.7, label=f"State-{i+1}
            ")
    #Labelling subplots
    ax1.set_xlabel("$N_A$")
    ax1.set_ylabel("Entanglement Entropy")
    ax1.set_title("Rényi-2")
    ax1.grid()
    ax2.set_xlabel("$N_A$")
    ax2.set_ylabel("Entanglement Entropy")
    ax2.set_title("von-Neumann")
    ax2.grid()
    ax1.legend()
    ax2.legend()
    if title!=None:
        plt.suptitle(title)
    plt.show()

def fin_size_study(N_vals, no_states, seed=None, title=None):
    """
    - This function plots the peak entropies (at  $N/2$ ) for each state
    - family, and compares them to the theoretical maximum  $N_A \ln(2)$ .
    - N_vals: List, contains INTEGERS for each system size  $N$  to
    - measure.
    - no_states: Integer, Number of states generated per value of  $N$ 
    - to average across.
    - seed: Integer, used for reproducibility (None by default)
    - title: Strings, gives plot a title (None by default)
    - """
    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(15,5)) #Subplots
    #Empty lists to store mean/deviations of each state family

```

```

Haar_S2_max, Haar_S_max = [], []
Cliff_S2_max, Cliff_S_max = [], []
conserve_S2_max, conserve_S_max = [], []
for N in N_vals:
    #Haar states
    Haar_states = Haar_random_generator(no_states=no_states, N
                                         =N, seed=seed) #Haar-random state sample
    Haar_S2, Haar_S, -, - = page_curve_vals(N=N, state_list=
        Haar_states, Rényi=True, von_Neumann=True, std=False) #
        Haar-random mean values for N
    #Appending values at N/2 to results
    Haar_S2_max.append(Haar_S2[N//2 - 1]) #-1 due to Python
        indexing
    Haar_S_max.append(Haar_S[N//2 - 1])
    #Clifford states
    Cliff_states = Clifford_stab_generator_v3(no_states=
        no_states, N=N, seed=seed) #Clifford state sample
    Cliff_S2, Cliff_S, -, - = page_curve_vals(N=N, state_list=
        Cliff_states, Rényi=True, von_Neumann=True, std=False) #
        Clifford mean values for N
    #Appending values at N/2 to results
    Cliff_S2_max.append(Cliff_S2[N//2 - 1])
    Cliff_S_max.append(Cliff_S[N//2 - 1])
    #Number-conserving states
    conserve_states = number_conserve_state_generator(
        no_states=no_states, N=N, n=N//2, seed=seed) #Number-
        conserving state sample
    conserve_S2, conserve_S, -, - = page_curve_vals(N=N,
        state_list=conserve_states, Rényi=True, von_Neumann=True
        , std=False)#Number-conserving mean values for N
    #Appending values at N/2 to results
    conserve_S2_max.append(conserve_S2[N//2 - 1])
    conserve_S_max.append(conserve_S[N//2 - 1])
    #Plotting everything
    S_max_vals = [(N//2)*np.log(2) for N in N_vals]
    #Plotting maximum S=N_A ln(2)
    ax1.plot(N_vals, S_max_vals, label="$S_{\max}=N_A \ln(2)$",
        color="red", ls="—")
    ax2.plot(N_vals, S_max_vals, label="$S_{\max}=N_A \ln(2)$",
        color="red", ls="—")
    #von Neumann plot for all families
    ax2.plot(N_vals, Haar_S_max, label="Haar-Random")
    ax2.plot(N_vals, Cliff_S_max, label="Cliffords")
    ax2.plot(N_vals, conserve_S_max, label="Number-Conserving")
    #Rényi-2 plot for all families

```



```

ax1.plot(N_vals, Haar_S2_max, label="Haar-Random")
ax1.plot(N_vals, Cliff_S2_max, label="Cliffords")
ax1.plot(N_vals, conserve_S2_max, label="Number-Conserving")
#Labelling axes and applying grid/legend
ax1.set_title("Rényi-2-Entropy:")
ax2.set_title("von-Neumann-Entropy:")
ax1.set_xlabel("N-(System-Size)")
ax2.set_xlabel("N-(System-Size)")
ax1.set_ylabel("Entanglement-Entropy")
ax2.set_ylabel("Entanglement-Entropy")
ax1.legend()
ax2.legend()
ax1.grid()
ax2.grid()
#Optional Title
if title!=None:
    plt.suptitle(title)
plt.show()

```

```

#
#                                     PLOTS
#

```

```

#Haar-Curve Plots: Plots the Haar page curve for N=6,8,12,16, and
#gives the mean/standard deviation of both entropies at N/2
if Haar_curve_plots:
    hcp_seed=42 #hcp_seed=42 used for report plots (the meaning to
    #life, the universe and everything)
    print("6-qubit-values-at-N_A=3")
    Haar_states_6 = Haar_random_generator(no_states=1000, N=6,
    seed=hcp_seed)
    plot_page_curve(N=6, state_list=Haar_states_6, Rényi=True,
    von-Neumann=True, errors=True,
    title="Haar-Random-States:-N=6",
    print_peak_vals=True) #6-qubit plot
    print("\n", "8-qubit-values-at-N_A=4")
    Haar_states_8 = Haar_random_generator(no_states=1000, N=8,
    seed=hcp_seed)
    plot_page_curve(N=8, state_list=Haar_states_8, Rényi=True,
    von-Neumann=True, errors=True,
    title="Haar-Random-States:-N=8",
    print_peak_vals=True) #8-qubit plot
    print("\n", "12-qubit-values-at-N_A=6")
    Haar_states_12 = Haar_random_generator(no_states=1000, N=12,
    seed=hcp_seed)

```

```

plot_page_curve(N=12, state_list=Haar_states_12, Rényi=True,
    von_Neumann=True, errors=True,
    title="Haar-Random States: -N=12",
    print_peak_vals=True) #12-qubit plot
print("\n", "16-qubit values at -N_A=8")
Haar_states_16 = Haar_random_generator(no_states=1000, N=16,
    seed=hcp_seed)
plot_page_curve(N=16, state_list=Haar_states_16, Rényi=True,
    von_Neumann=True, errors=True,
    title="Haar-Random States: -N=16",
    print_peak_vals=True) #No errors since it's
    not visible for 16 qubits.

print("\n")

#Conserve Curve Plots: Plots the Haar page curve for N=6,8,12,16,
    and gives the mean/standard deviation of both entropies at N/2
if conserve_curve_plots:
    ccp_seed=42 #ccp_seed=42 used for report plots
    print("6-qubit values at -N_A=3")
    conserve_states_6 = number_conserve_state_generator(no_states
        =1000, N=6, n=3, seed=ccp_seed)
    plot_page_curve(N=6, state_list=conserve_states_6, Rényi=True,
        von_Neumann=True, errors=True,
        title="Number-Conserving States: -N=6, -n=3") #
        6-qubit plot
    print("\n", "8-qubit values at -N_A=4")
    conserve_states_8 = number_conserve_state_generator(no_states
        =1000, N=8, n=4, seed=ccp_seed)
    plot_page_curve(N=8, state_list=conserve_states_8, Rényi=True,
        von_Neumann=True, errors=True,
        title="Number-Conserving States: -N=8, -n=4") #
        8-qubit plot
    print("\n", "12-qubit values at -N_A=6")
    conserve_states_12 = number_conserve_state_generator(no_states
        =1000, N=12, n=6, seed=ccp_seed)
    plot_page_curve(N=12, state_list=conserve_states_12, Rényi=
        True, von_Neumann=True, errors=True,
        title="Number-Conserving States: -N=12, -n=6") #
        12-qubit plot
    print("\n", "16-qubit values at -N_A=8")
    conserve_states_16 = number_conserve_state_generator(no_states
        =1000, N=16, n=8, seed=ccp_seed)
    plot_page_curve(N=16, state_list=conserve_states_16, Rényi=
        True, von_Neumann=True, errors=True,
        title="Number-Conserving States: -N=16, -n=8") #

```

```

                                No errors since it's not visible for 16
                                qubits.

print("\n")

#Clifford Curve Plots: Plots the Clifford Page curves for N
                        =6,8,12,16, and gives the mean/standard deviation of both
                        entropies at N/2
if Clifford_curve_plots:
    Cliff_seed = 42 #Cliff_seed=42 used for report plots
    print("6-qubit-values-at-N_A=3")
    Clifford_sample_6 = Clifford_stab_generator_v3(no_states=1000,
        N=6, seed=Cliff_seed) #Used for mean entropy plot, 6 qubits
    plot_page_curve(N=6, state_list=Clifford_sample_6, Rényi=True,
        von_Neumann=True, errors=True,
        title="Clifford-Stabilizer-States:-N=6") #6-
        qubit plot
    print("\n", "8-qubit-values-at-N_A=4")
    Clifford_sample_8 = Clifford_stab_generator_v3(no_states=1000,
        N=8, seed=Cliff_seed) #Used for mean entropy plot, 6 qubits
    plot_page_curve(N=8, state_list=Clifford_sample_8, Rényi=True,
        von_Neumann=True, errors=True,
        title="Clifford-Stabilizer-States:-N=8") #8-qubit plot
    print("\n", "12-qubit-values-at-N_A=6")
    Clifford_sample_12 = Clifford_stab_generator_v3(no_states
        =1000, N=12, seed=Cliff_seed) #Used for mean entropy plot,
        12 qubits
    plot_page_curve(N=12, state_list=Clifford_sample_12, Rényi=
        True, von_Neumann=True, errors=True,
        title="Clifford-Stabilizer-States:-N=12") #12-
        qubit plot
    #This plot in particular takes a while, but does work.
    print("\n", "16-qubit-values-at-N_A=8")
    Clifford_sample_16 = Clifford_stab_generator_v3(no_states
        =1000, N=16, seed=Cliff_seed) #Used for mean entropy plot,
        16 qubits
    plot_page_curve(N=16, state_list=Clifford_sample_16, Rényi=
        True, von_Neumann=True, errors=True,
        title="Clifford-Stabilizer-States:-N=16") #16-
        qubit plot

    print("\n")

#Indiv Clifford Plots: Plots the individual entropies of a 6
                        Clifford stabilizer states, for N=12
if indiv_Clifford_plots:
    seed_0=48 #seed_0=48 used in report

```

```

Clifford_sample_indiv = Clifford_stab_generator_v3(no_states
    =6, N=12, seed=seed_0) #Used for individual entropies
page_curve_indiv_state_entropy_comparison_subplots(N=12,
    state_list=Clifford_sample_indiv) #Indiv plot

#S_max Comparison: This tests plots the entanglement entropies of
    200 of each state family and compares them to the theoretical
    maximum  $N_A \ln(2)$ .
#Here, we specifically only consider the von Neumann entropies, as
    for each state family the Rényi-2 entropy is less than or equal
    to the von Neumann entropy.
if S_max_comparison:
    seed_1=42 #seed_1=42 used in report
    #First plot, compares the Page Curves of samples of 100 states
        from each state family in a 8-qubit system. (including  $N_A$ 
         $\ln(2)$ )
    state_samples_1 = [Haar_random_generator(no_states=500, N=8,
        seed=seed_1, print_ans=False),
        Clifford_stab_generator_v3(no_states=500, N
            =8, print_ans=False, seed=seed_1),
        number_conserve_state_generator(no_states
            =500, N=8, n=4, seed=seed_1)]
    page_curve_state_comparison_plot(N=8, list_of_families =
        state_samples_1, family_names=["Haar-Random", "Clifford", "
        Number-Conserving"], Rényi=False, von_Neumann=True, S_max=
        True, title="Page-Curves-vs-Maximum-Entropy-(von-Neumann):-N
        =8", label_entropy=False)
    #Second plot, does the same for 16 qubits.
    state_samples_2 = [Haar_random_generator(no_states=500, N=16,
        seed=seed_1, print_ans=False), Clifford_stab_generator_v3(
        no_states=500, N=16, print_ans=False, seed=seed_1),
        number_conserve_state_generator(no_states=500, N=16, n=8,
        seed=seed_1)]
    page_curve_state_comparison_plot(N=16, list_of_families =
        state_samples_2, family_names=["Haar-Random", "Clifford", "
        Number-Conserving"], Rényi=False, von_Neumann=True, S_max=
        True, title="Page-Curve-vs-Maximum-Entropy-(von-Neumann):-N
        =16", label_entropy=False)

#Finite Size Study: This test compares the 'peak' of each
    entanglement distribution (i.e. taking  $N/2$ )
if peak_comparison:
    seed_2=42 #seed_2=42 used for report
    fin_size_study(N_vals=[4,6,8,10], no_states=1000, seed=seed_2)

```

7.4 File 4 (Superposition Analysis):

```
"""
This file contains all the code to plot  $\langle S_2(1) \rangle$ ,  $S_2(m)$  and  $dS_2(m)$ , including plotting functions to compare each quantity between different state families, and different sizes of  $N$ . This corresponds to Sections 3.6 and 4.2 of the report.
"""

import numpy as np
import matplotlib.pyplot as plt
from RDM_Entropy_Calcs_and_Validation import reduced_density_matrix, Rényi_2_entropy
from State_Gens import Haar_random_generator, number_conserve_state_generator, Clifford_stab_generator_v3

S_vs_m_system_size_study = False #If True, plots  $\langle S_2(1) \rangle$ ,  $S_2(m)$  and  $dS_2(m)$  in separate subplots for each state family as presented in report.
#(i.e Haar-random states have 3 subplots as part of one larger plot, Clifford and number-conserving plots give 3 separate figures each)
Haar_single_state_vs_Clifford_superpos = False #If True, this plots the Haar-random mean single-state entropy, and the Clifford superposition entropy
#(the latter should approximately converge to the former)
number_conserve_vs_log_m = False #If True, plots  $dS_2(m)$  against  $\ln(m)$  (highlights the non-logarithmic nature of  $dS_2$  for number-conserving states)
dS_state_comparison = False #If True, plots the mean single-state entropy, the superposition entropy and their difference in 3 separate subplots
#(each subplot contains the given quantity for all 3 state families)

#=====
#      SUPPORTING FUNCTIONS TO CALCULATE  $dS_2$  FOR PLOTTING
#=====

def equal_weight_superposition(state_list, print_ans=False):
    """
    --- Creates an equal sum of a sample of states, and then normalizes it.
    --- state_list: Collection of states to be summed (ensure this is a list of Numpy Arrays?)
    """
```

```

---- print_ans: Boolean, print answer if True
---- """
    total_sum = sum(state_list) #Sum all states
    superpos = total_sum / np.linalg.norm(total_sum) #Normalize
    the sum
    if print_ans: #Print the superposition if True
        print(superpos)
    return superpos

def mean_Rényi_entropy(state_list, N, qubits_kept, print_ans=False):
    """
    ---- Calculates the mean Rényi-2 entropy for a given list of states
    .
    ---- state_list: List, contains the states to measure mean entropy
    of
    ---- N: Integer, number of qubits
    ---- qubits_kept: List, contains integers to represent qubits NOT
    traced out (starts from 0=1st qubit, etc.)
    ---- print_ans: Boolean, prints answer if True
    ---- """
    entropies=[] #List for the entropy of each state
    for state in state_list:
        rho = reduced_density_matrix(vec=state, qubits_kept=
            qubits_kept, N=N, print_ans=False) #Reduced density
            matrix
        S2 = Rényi_2_entropy(rho, print_ans=False) #Entropy of 1
            state
        entropies.append(S2)
    mean_S2 = np.mean(entropies) #Mean Rényi-2 entropy
    if print_ans: #Print mean entropy if True
        print(mean_S2)
    return mean_S2

def superposition_Rényi_entropy(state_list, N, qubits_kept,
    print_ans=False):
    """
    ---- Calculates the Rényi-2 entropy for the equal-weight
    superposition of a given list of states.
    ---- state_list: List, contains the states to measure superposition
    entropy of
    ---- N: integer, number of qubits
    ---- qubits_kept: List, contains integers to represent qubits NOT
    traced out (starts from 0=1st qubit, etc.)
    ---- print_ans: Boolean, prints answer if True

```

```

"""
    superpos = equal_weight_superposition(state_list=state_list ,
        print_ans=False) #Superposition
    superpos_rdm = reduced_density_matrix(vec=superpos ,
        qubits_kept=qubits_kept , N=N, print_ans=False) #
    Superposition RDM
    superpos_S2 = Rényi_2_entropy(rho=superpos_rdm , print_ans=
        False) #Entropy
    if print_ans: #Print entropy if True
        print(superpos_S2)
    return superpos_S2

def Rényi_plot_vals_one_step(N, state_list , qubits_kept , print_ans
    =False):
    """
    ----- Calculates  $dS_2(m) = S_2(m) - \langle S_2(1) \rangle$ , then  $S_2(m)$ , and finally  $\langle S_2(1) \rangle$ . Used to get the values for plots of  $dS_2$  against  $m$ , this gives the values needed FOR A SINGLE VALUE OF  $M$ 
    -----  $N$ : Integer , total number of qubits
    ----- state_list: List , contains family of states to test
    ----- qubits_kept: List , contains all INTEGERS of qubits to keep (
        START-AT 0)
    ----- print_ans: Boolean , print values if True ( False by default)
    ----- """
    S2_superpos = superposition_Rényi_entropy(state_list , N,
        qubits_kept , print_ans=False) #Rényi-2 entropy of
        superposition
    S2_mean = mean_Rényi_entropy(state_list=state_list , N=N,
        qubits_kept=qubits_kept , print_ans=False) #Average entropy
        of all states individually
    dS2 = S2_superpos - S2_mean #Difference
    if print_ans:
        print("dS2:", dS2, " Superposition Entropy:-", S2_superpos ,
            " Mean Entropy of Individual States:-", S2_mean)
    return dS2, S2_superpos , S2_mean

def smart_state_generator(state_gen , no_states , N, n=None, seed=
    None):
    """
    ----- This function chooses the correct state family generator based
        on a string inputted , and generates the states needed .
    ----- This can be amended with any other state generators needed .
    ----- state_gen: String , determines which generator to use from
        State_Gens.py to build states .
    ----- Current accepted inputs:

```

```

-----"Haar": Generates Haar-random states
-----"Clifford": Generates Clifford stabilizer states
-----"Conserve": Generates number-conserving free fermion
states.
----no_states: Integer, Number of states to generate
----N: Integer, number of qubits in the system
----n: Number of fermions (only relevant for number-conserving
states) (None by default)
----MAKE SURE n HAS A VALUE FOR NUMBER-CONSERVING STATES
-----"""
    if state_gen=="Haar":
        state_list = Haar_random_generator(no_states=no_states, N=
            N, print_ans=False, seed=seed) #Haar-random states
    elif state_gen=="Clifford":
        state_list = Clifford_stab_generator_v3(no_states=
            no_states, N=N, seed=seed) #Clifford states
    elif state_gen=="Conserve":
        state_list = number_conserve_state_generator(no_states=
            no_states, N=N, n=n, print_ans=False, seed=seed) #Number
            -conserving states
    else:
        return("Invalid state generator.") #Safety check
    return state_list

```

```

=====
#                               dS2 VALUE CALCULATOR (TO USE FOR PLOTS)
=====

```

```

def dS2_vs_m_values(N, qubits_kept, no_trials, state_gen, m_vals,
    seed=None, std=True):
    """
    ----This function generates the values of the mean single-state
    entropy, the superposition entropy, and their difference dS2 for
    all values of m in m_vals. These values can be used to plot
    each quantity as desired.
    ----Returns the values of --(in order): The mean single-state-
    entropy, its standard deviation, the superposition entropy, its
    deviation, dS2, its deviation, for all values of m. (i.e. each
    one of these is a list, containing their values for all values
    of m tested)
    ----N: Integer, number of qubits in system
    ----qubits_kept: List, contains integers for qubits in the tested
    subsystem (counting from 0)
    ----no_trials: Integer, number of random state samples tested.
    ----state_gen: String, determines which generator to use from

```



```

State_Gens.py to build states.
-----Current accepted inputs:
-----"Haar": Generates Haar-random states
-----"Clifford": Generates Clifford stabilizer states
-----"Conserve": Generates number-conserving free fermion
states.
----m_vals: List, contains integers of all m values to take.
----seed: Integer, used for reproducibility (None by default)
----std: Boolean, calculates standard deviations too if True (True
by default) (returns empty lists otherwise)
----"""

dS2_vals = [] #List for average dS2 results
indiv_S2_vals = [] #List for mean entropy of individual state
superpos_S2_vals = [] #List for equal-weight superposition
entropy_vals
m_max = max(m_vals) #Maximum m value
for _ in range(no_trials):
    #Optional check: if seed non-empty, add 1 to seed each
    instance (allows reproducibility, ensures each trial has
    distinct states)
    if seed!=None:
        seed+=1
    dS2_trial_vals = [] #List for all dS2 trial values
    m_indiv_S2_trials = [] #List for all mean S2 trial values
    m_superpos_S2_trials = [] #List for all superposition S2
    trial values
    state_sample = smart_state_generator(state_gen=state_gen,
        no_states=m_max, N=N, n=N//2)
    #State sample = m_max states generated once per trial (
    more efficient/useful than regenerating whole list for
    each m value)
    np.random.shuffle(state_sample) #Makes sure no state is
    oversampled
    for m in m_vals: #Loop repeated for each step between
    m_min, m_max
        m_val_states = state_sample[:m] #First m terms of
        state_sample used for test
        dS2_m, superpos_S2, indiv_S2 = Ré
        nyi_plot_vals_one_step(N=N, state_list=m_val_states,
            qubits_kept=qubits_kept, print_ans=False)
        #Adding each step with m to the trial results
        dS2_trial_vals.append(dS2_m)
        m_superpos_S2_trials.append(superpos_S2)
        m_indiv_S2_trials.append(indiv_S2)
    #Adding the trial results to the overall results

```

```

    dS2_vals.append(dS2_trial_vals)
    superpos_S2_vals.append(m_superpos_S2_trials)
    indiv_S2_vals.append(m_indiv_S2_trials)
#Finding the means/standard deviations of all trial results:
#dS2 Means and deviations
dS2_array = np.array(dS2_vals) #Shape (no_trials , len(m_vals))
dS2_mean = np.mean(dS2_array , axis=0) #Mean of each column of
    dS2 array (i.e. the mean dS2 per value of m)
dS2_stds = np.std(dS2_array , axis=0) if std else None #dS2
    standard deviations per value of m
#Mean and deviation of individual entropies
indiv_S2_array = np.array(indiv_S2_vals) #Shape (no_trials ,
    len(m_vals))
indiv_S2_mean = np.mean(indiv_S2_array , axis=0)
indiv_S2_stds = np.std(indiv_S2_array , axis=0) if std else
    None #individual mean entropies standard deviations per
    value of m
#Mean and deviation of superposition entropy
superpos_S2_array = np.array(superpos_S2_vals)
superpos_S2_mean = np.mean(superpos_S2_array , axis=0)
superpos_S2_stds = np.std(superpos_S2_array , axis=0) if std
    else None #Superposition standard deviations per value of m
return indiv_S2_mean , indiv_S2_stds , superpos_S2_mean ,
    superpos_S2_stds , dS2_mean , dS2_stds

```

```

//=====
//                                PLOTTING FUNCTIONS
//=====

```

```

def dS2_vs_m_plot_v3(N, qubits_kept , state_gen , m_min, m_max,
    m_step , no_trials , seed=None, std=True):
    """
    ----THIS IS NOT USED IN THE FINAL REPORT.
    ----For a given method of state-generation , this function plots
        the mean single-state entropy and the equal-weight superposition
        Rényi-2 entropies against the number of states m on a plot on
        the left , and the difference between the two against m on a plot
        on the right .
    ----N: Integer , number of qubits
    ----qubits_kept: List of integers , denote to qubits to keep in the
        subsystem
    ----state_gen: String , determines which generator to use from
        State_Gens.py to build states .
    -----Current accepted inputs:
    -----"Haar": Generates Haar-random states
    """

```

```

-----" Clifford": Generates Clifford stabilizer states
-----" Conserve": Generates number-conserving free fermion
states.
----m_min, m_max: Integers, denote the minimum/maximum no of
states taken from sample
----m_step: Integer, number of states added at each interval
----no_trials: Integer, determines how many sets of random states
are generated per step of m
----seed: Integer, seed for random sampling from state_list (None
by default)
----std: Boolean, adds standard deviation region to plot if True (
True by default)
-----"""

m_vals = np.arange(m_min, m_max+1, m_step) #m_max+1 since np.
arange gives [start, stop)
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(15,5)) #Subplots
(ax1 plots mean indiv and superposition entropies, ax2
plots dS2)
indiv_S2_mean, indiv_S2_stds, superpos_S2_mean,
superpos_S2_stds, dS2_mean, dS2_stds = dS2_vs_m_values(N=N,
qubits_kept=qubits_kept, no_trials=no_trials, state_gen=
state_gen, m_vals=m_vals, seed=seed, std=std) #Finding
the values for everything
#Plot Everything
ax2.plot(m_vals, dS2_mean, label="$\Delta S_2$", color="green"
) #dS2 values
ax1.plot(m_vals, superpos_S2_mean, label=r"$S_2(m)$", color="
red") #Plot of superposition entropies
ax1.plot(m_vals, indiv_S2_mean, label=r"$\langle S_2(1) \rangle$
", color="blue") #Plot of indiv entropies
if std:
    ax2.fill_between(m_vals, np.add(dS2_mean, dS2_stds), np.
subtract(dS2_mean, dS2_stds), color="green", alpha=0.2)
#Standard deviation of dS2 above
    ax1.fill_between(m_vals, np.add(indiv_S2_mean,
indiv_S2_stds), np.subtract(indiv_S2_mean, indiv_S2_stds
),
color="blue", alpha=0.2) #Standard deviation
of mean entropy of individual states
    ax1.fill_between(m_vals, np.add(superpos_S2_mean,
superpos_S2_stds), np.subtract(superpos_S2_mean,
superpos_S2_stds),
color="red", alpha=0.2) #Standard deviation
of equal weight superposition states
#Labelling axes:

```

```

ax1.set_xlabel("m (Number of States)")
ax2.set_xlabel("m (Number of States)")
ax1.set_ylabel("Entanglement Entropy")
ax2.set_ylabel("Entanglement Entropy")
ax1.set_title("Components of  $\Delta S_2(m)$ ")
ax2.set_title(" $\Delta S_2(m)$ ")
#Grids for both
ax1.grid()
ax2.grid()
ax1.legend()
plt.show()

def comparison_dS2_for_all_state_classes(N, qubits_kept, m_min,
    m_max, m_step, no_trials, seed=None, std=True):
    """
    ----- This function plots  $\langle S_2(1) \rangle$ ,  $S_2(m)$  and  $dS_2(m)$  for all 3-
        state families, with each quantity in a separate subplot (and
        each subplot containing all 3 state families at once.)
    ----- This can be amended with more classes if desired.
    ----- N: Integer, total number of qubits in system
    ----- qubits_kept: List, contains INTEGERS of qubits not to be
        traced out for reduced density matrices when calculating
    ----- m_min, m_max: Integers, denote the minimum/maximum no of
        states taken from sample
    ----- m_step: Integer, number of states added at each interval
    ----- no_trials: Integer, determines how many trials of random
        states are generated per step of m
    ----- seed: Integer, seed for random sampling from state_list (None
        by default)
    ----- std: Boolean, include standard deviations for everything if
        True (True by default)
    ----- """
    state_classes = ["Haar", "Clifford", "Conserve"] #Can be
        modified with more state classes if needed.
    colours = ["blue", "red", "green"] #Must have at least the
        same amount of elements as state_classes
    state_names = ["Haar-Random", "Clifford", "Number-Conserving"]
        #Must have at least the same amount of elements as
        state_classes
    fig, (ax1, ax2, ax3) = plt.subplots(3,1, figsize=(10,12))
    #ax1 plots the mean single-state entropy, ax2 plots the
        superposition entropy, ax3 plots dS2
    m_vals = np.arange(m_min, m_max+1, m_step) #m_max+1 since np.
        arange gives [start, stop)
    for state_class, colour, state_name in zip(state_classes,

```

```

colours, state_names):
    #Get all values for each state class.
    indiv_S2_mean, indiv_S2_stds, superpos_S2_mean,
    superpos_S2_stds, dS2_mean, dS2_stds = dS2_vs_m_values(N
    =N,
    qubits_kept=qubits_kept, no_trials=no_trials,
    state_gen=state_class, m_vals=m_vals, seed=seed, std
    =True)
    #Plot the means of everything
    ax1.plot(m_vals, indiv_S2_mean, label = state_name, color=
    colour) #Mean single-state entropy (1st plot)
    ax2.plot(m_vals, superpos_S2_mean, label = state_name,
    color=colour) #Superposition entropy (1st plot)
    ax3.plot(m_vals, dS2_mean, label=state_name, color=colour)
    #dS2 values
    #Plots standard deviations only if std==True:
    if std:
        ax1.fill_between(m_vals, np.add(indiv_S2_mean,
        indiv_S2_stds), np.subtract(indiv_S2_mean,
        indiv_S2_stds), color=colour, alpha=0.2)
        ax2.fill_between(m_vals, np.add(superpos_S2_mean,
        superpos_S2_stds), np.subtract(superpos_S2_mean,
        superpos_S2_stds), color=colour, alpha=0.2)
        ax3.fill_between(m_vals, np.add(dS2_mean, dS2_stds),
        np.subtract(dS2_mean, dS2_stds), color=colour, alpha
        =0.2)
    #Labelling plot and adding grids/legends
    ax1.set_ylabel(r"$\langle S_2(1) \rangle$ - (Mean Single-State
    Entropy)")
    ax2.set_ylabel(r"$S_2(m)$ - (Superposition Entropy)")
    ax3.set_xlabel("m - (Number of States)")
    ax3.set_ylabel("$\Delta S_2(m)$")
    ax1.grid()
    ax2.grid()
    ax3.grid()
    ax1.legend()
    ax2.legend()
    ax3.legend()
    plt.show()

def Haar_vs_clifford_plot(N, qubits_kept, no_trials, m_min, m_max,
m_step, seed=None, title=None, std=False):
    """
    ---- This function is used to directly compare the Haar-random
    single-state entropy and the Clifford stabilizer superposition

```

```

    entropy.
    """N: Integer, number of qubits
    """qubits_kept: List, contains integers of qubits in the
        subsystem kept
    """no_trials: Integer, number of lists of random states tested (
        m_max states generated each time)
    """m_min, m_max: Integers, minimum and maximum number of states
        taken from sample
    """m_step: Integer, determines how size of state samples scales (
        m_step added each time)
    """seed: Integer, used for reproducibility (None by default)
    """title: String, gives the plot a title (None by default)
    """std: Boolean, includes standard deviations if True (False by
        default)
    """
    m_vals = np.arange(m_min, m_max+1, m_step)
    #Haar-random single-state plot
    H_indiv_S2_mean, H_indiv_S2_stds, _, _, _ = dS2_vs_m_values
        (N=N,
         qubits_kept=qubits_kept, no_trials=no_trials, state_gen="
         Haar", m_vals=m_vals, seed=seed, std=True) #Mean/
         deviation of Haar-random single-state entropy.
    plt.plot(m_vals, H_indiv_S2_mean, label=r"Haar-random Single-
        State-$S_2$", color="blue") #Plot mean of Haar-random single
        -state entropy
    #Clifford superposition plot
    _, _, Cliff_superpos_mean, Cliff_superpos_stds, _, _ =
        dS2_vs_m_values(N=N,
         qubits_kept=qubits_kept, no_trials=no_trials, state_gen="
         Clifford", m_vals=m_vals, seed=seed, std=True) #Mean/
         deviation of Clifford superposition entropy.
    plt.plot(m_vals, Cliff_superpos_mean, label=r"Clifford-
        Superposition-$S_2$", color="red") #Plot mean of Clifford
        superposition entropy.
    if std: #Plot deviations of both if std==True
        plt.fill_between(m_vals, np.add(H_indiv_S2_mean,
            H_indiv_S2_stds), np.subtract(H_indiv_S2_mean,
            H_indiv_S2_stds), color="blue", alpha=0.2)
        plt.fill_between(m_vals, np.add(Cliff_superpos_mean,
            Cliff_superpos_stds), np.subtract(Cliff_superpos_mean,
            Cliff_superpos_stds), color="red", alpha=0.2)
    plt.xlabel("m (Number of States)")
    plt.ylabel("Entanglement Entropy")
    if title!=None:
        plt.title(title)

```

```

plt.legend()
plt.grid()
plt.show()

def dS2_vs_m_N_comparison(N_vals, qubits_kept_list, state_gen,
                           m_min, m_max, m_step, no_trials,
                           seed=None, std=True, subplots=
                               False):
    """
    -----For a given state generation, this function plots <S_2(1)
    >, S_2(m) and dS_2(m) for multiple values of N.
    -----This function will can either have separate plots for each
    quantity, or have them as subplots in a singular plot.
    -----N_vals: Lists, contains integers for each system size
    studied.
    -----state_gen: String, determines which generator to use from
    Part2v2 to build states.
    -----Current accepted inputs:
    -----"Haar": Generates Haar-random states
    -----"Clifford": Generates Clifford stabilizer states
    -----"Conserve": Generates number-conserving free-
    fermion states.
    -----qubits_kept_list: List, contains multiple LISTS, each of
    which contains the integers kept when finding the reduced
    -----density matrix. Note 0=1st qubit, 1=2nd qubit, etc,
    and that the order of the lists must match the order of the
    -----numbers in N_vals (e.g. the first list of qubits_kept
    corresponds to the first value of N_vals taken, etc.)
    -----m_min, m_max: Integers, denote the minimum/maximum no of
    states taken from sample
    -----m_step: Integer, number of states added at each interval
    -----no_trials: Integer, determines how many sets of random
    states are generated per step of m
    -----seed: Integer, seed for random sampling from state_list
    -----std: Boolean, plots standard deviation if True (True by
    default)
    -----subplots: Boolean, plots each quantity as subplots on a
    single plot if True (otherwise each plot is returned separately)
    -----"""
    colours=["#1f77b4", "#ff7f0e", "#2ca02c", "#d62728"] #
    Colours listed here to ensure means/deviations match (
    this list must have more entries than number of N vals,
    or it breaks!)
    m_vals = np.arange(m_min, m_max+1, m_step) #m_max+1 since
    np.arange gives [start, stop)

```

```

if subplots:
    fig, (ax1, ax2, ax3) = plt.subplots(3, 1, figsize
        =(7.5,15)) #Subplots in the same figure(ax1 plots
        mean indiv and superposition entropies, ax2 plots
        dS2)
else: #Used to get separate plots for report.
    fig1, ax1 = plt.subplots(1, 1, figsize=(7.5, 5))
    fig2, ax2 = plt.subplots(1, 1, figsize=(7.5, 5))
    fig3, ax3 = plt.subplots(1, 1, figsize=(7.5, 5))
#Iterating for each value of N given:
for i, N in enumerate(N_vals): #Testing each value of N, i
    is the index of N_vals
    indiv_S2_mean, indiv_S2_stds, superpos_S2_mean,
    superpos_S2_stds, dS2_mean, dS2_stds =
    dS2_vs_m_values(N=N,
        qubits_kept=qubits_kept_list[i], no_trials=
        no_trials, state_gen=state_gen, m_vals=m_vals,
        seed=seed, std=std)
#Plot Everything
ax3.plot(m_vals, dS2_mean, label=f"N={N}", color=
    colours[i]) #dS2 values
ax2.plot(m_vals, superpos_S2_mean, label=f"N={N}",
    color=colours[i]) #Plot of superposition entropies
ax1.plot(m_vals, indiv_S2_mean, label=f"N={N}", color=
    colours[i]) #Plot of indiv entropies
if std:
    ax3.fill_between(m_vals, np.add(dS2_mean, dS2_stds
        ), np.subtract(dS2_mean, dS2_stds), color=
        colours[i], alpha=0.2)
#Standard deviation of dS2 above
ax1.fill_between(m_vals, np.add(indiv_S2_mean,
    indiv_S2_stds), np.subtract(indiv_S2_mean,
    indiv_S2_stds),
        color=colours[i], alpha=0.2) #Standard
        deviation of mean entropy of
        individual states
ax2.fill_between(m_vals, np.add(superpos_S2_mean,
    superpos_S2_stds), np.subtract(superpos_S2_mean,
    superpos_S2_stds),
        color=colours[i], alpha=0.2) #Standard
        deviation of equal weight superposition
        states
#Labelling axes:
ax1.set_xlabel("m (Number of States)")
ax2.set_xlabel("m (Number of States)")

```



```

ax3.set_xlabel("m (Number of States)")
ax1.set_ylabel(r"$\langle S_2(1) \rangle$ (Mean Single-  
State Entropy)")
ax2.set_ylabel(r"$S_2(m)$ (Superposition Entropy)")
ax3.set_ylabel(r"$\Delta S_2(m)$")
#Grids and legends for all
ax1.grid()
ax2.grid()
ax3.grid()
ax1.legend()
ax2.legend()
ax3.legend()
plt.show()

def dS2_vs_ln_m(N, qubits_kept, state_gen, m_min, m_max, m_step,
               no_trials, seed=None, std=True):
    """
    -----This function plots dS2-ONLY, and plots this against ln(m)
    .- Similar to dS_vs_m_v3, but only with one plot.
    -----Used to show number-conserving states do not have
    logarithmic growth.
    -----N: Integer, number of qubits
    -----qubits_kept: List of integers, denote to qubits to keep in
    the subsystem
    -----state_gen: String, determines which generator to use from
    State_Gens.py to build states.
    -----Current accepted inputs:
    -----"Haar": Generates Haar-random states
    -----"Clifford": Generates Clifford stabilizer states
    -----"Conserve": Generates number-conserving free-
    fermion states.
    -----m_min, m_max: Integers, denote the minimum/maximum no of
    states taken from sample
    -----m_step: Integer, number of states added at each interval
    -----no_trials: Integer, determines how many sets of random
    states are generated per step of m
    -----seed: Integer, seed for random sampling from state_list (
    None by default)
    -----std: Boolean, adds standard deviation region to plot if
    True (True by default)
    """
    m_vals = np.arange(m_min, m_max+1, m_step) #m_max+1 since
    np.arange gives [start, stop)
    -, -, -, -, dS2_mean, dS2_stds = dS2_vs_m_values(N=N,
    qubits_kept=qubits_kept, no_trials=no_trials, state_gen=

```

```

        state_gen, m_vals=m_vals, seed=seed, std=std) #dS2 mean
        and deviation
#Plot Everything
plt.plot(np.log(m_vals), dS2_mean, label="$\Delta S_2$",
        color="#1f77b4") #dS2 values
if std:
    plt.fill_between(np.log(m_vals), np.add(dS2_mean,
        dS2_stds), np.subtract(dS2_mean, dS2_stds), color=
        "#1f77b4", alpha=0.2)
        #Standard deviation of dS2 above
#Labelling axes:
plt.xlabel("ln(m)")
plt.ylabel("$\Delta S_2(m)$")
plt.grid()
plt.show()

#=====
#                               PLOTS
#=====

#S vs m System Size Study: Plots Haar-random, Clifford and number-
conserving states separately, each for N=4,6,8,10, with 500
    trials and m_max=50
#(number-conserving states take half-filling, i.e. n=N/2)
if S_vs_m_system_size_study: #These plots take a while.
    seed_1 = 42 #seed_1=42 used for report.
    #Haar-random plot
    dS2_vs_m_N_comparison(N_vals=[4,6,8,10], qubits_kept_list
       =[[0,1], [0,1,2], [0,1,2,3], [0,1,2,3,4]],
        state_gen="Haar", m_min=1, m_max=50, m_step=1, no_trials
            =500, std=True, seed=seed_1, subplots=True)
    #Clifford plot
    dS2_vs_m_N_comparison(N_vals=[4,6,8,10], qubits_kept_list
       =[[0,1], [0,1,2], [0,1,2,3], [0,1,2,3,4]],
        state_gen="Clifford", m_min=1, m_max=50, m_step=1,
            no_trials=500, std=True, seed=seed_1, subplots=False)
    #Number-conserving plot
    dS2_vs_m_N_comparison(N_vals=[4,6,8,10], qubits_kept_list
       =[[0,1], [0,1,2], [0,1,2,3], [0,1,2,3,4]],
        state_gen="Conserve", m_min=1, m_max=50, m_step=1,
            no_trials=500, std=True, seed=seed_1, subplots=False)

#Haar Single State vs Clifford Superpos: This directly compares
the Haar-random single-state entropy to the Clifford
superposition.

```

```

#Plots both for N=8, 500 trials , up to 50 states.
if Haar_single_state_vs_Clifford_superpos:
    seed_2=42 #seed_2=42 used for report.
    Haar_vs_clifford_plot(N=8, qubits_kept=[0,1,2,3], no_trials
        =500, m_min=1, m_max=50, m_step=1, seed=seed_2)

#Number-Conserving Log: Plots dS2 against ln(m) for number-
    conserving states , with N=8, 500 trials , up to 100 states.
if number_conserve_vs_log_m:
    seed_3=42 #seed_3 = 42 used for report.
    dS2_vs_ln_m(N=8, qubits_kept=[0,1,2,3], state_gen="Conserve",
        m_min=1, m_max=100, m_step=1, no_trials=500, seed=seed_3,
        std=True)

#dS State Comparison: Plots <S_2(1)>, S_2(m) and dS_2(m)=S_2(m) -
    <S_2(1)> for all 3 state classes , with each quantity in separate
    subplots.
#This is for N=8, with 500 trials per family and up to 50 states.
if dS_state_comparison:
    seed_4=42 #seed_4 = 42 used for report.
    comparison_dS2_for_all_state_classes(N=8, qubits_kept
        =[0,1,2,3], m_min=1, m_max=50, m_step=1, no_trials=500, seed
        =seed_4 , std=True)

```